

计算机体系结构

L02 存储层次

计算机体系结构

背景知识 (自学) ③

存储层次

内存层次概念

Cache

映射策略 (Placement) ③

替换策略 ⑤

写策略

命中时 ②

非命中 ②

提升Cache性能

提高命中率 ⑥

降低 Hit Latency ②

降低 Miss Penalty ②

其他方法

虚拟内存

地址转换

页表

TLB

地址转换过程优化

虚地址 Cache

TLB和Cache并行访问

单处理器 (指令级并行) ④

简单流水线 ④

复杂流水线 ②

分支预测 ③

指令并行高级技术 ②

多处理器 (线程级并行) ⑤

加速器 (数据并行) ③

存储层次基本概念

(Memory Hierarchy)

Memory技术

□ 早期计算机使用的memory技术多种多样

- Manchester Mark I 采用阴极射线管存储器(CRT Memory Storage)
- EDVAC 采用汞延迟线(mercury delay line)

□ 磁芯存储器(Core memory)是第一个大规模主存

- 由Forrester在1940年左右发明
- 将数据存储为磁化极性，存储在穿过二维网格的小铁氧体磁芯上

□ 第一个商用DRAM是Intel 1103

- 单片容量为 1Kbit
- 通过电容器上的电荷保持数值

□ 1970年左右半导体存储器迅速取代了磁芯存储器

- Intel成立，主攻半导体存储器

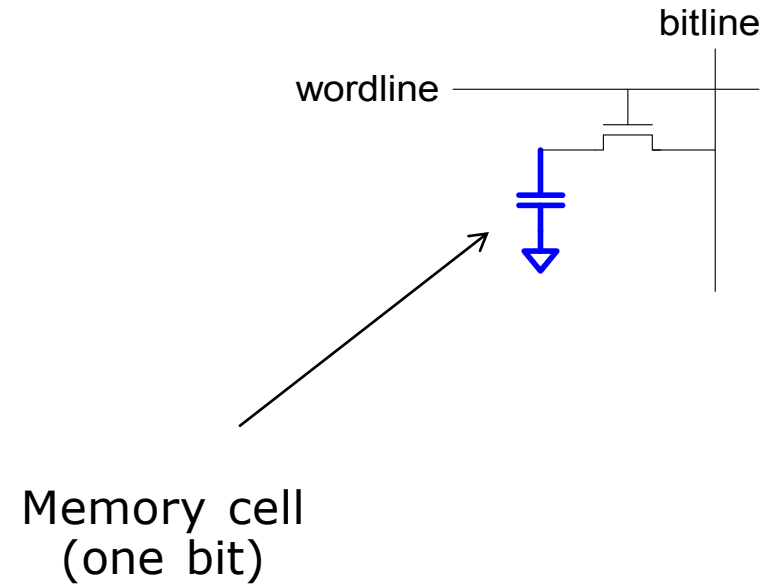
□ 闪存

- 更慢, 但密度比DRAM高. 非易失, 但有磨损的问题

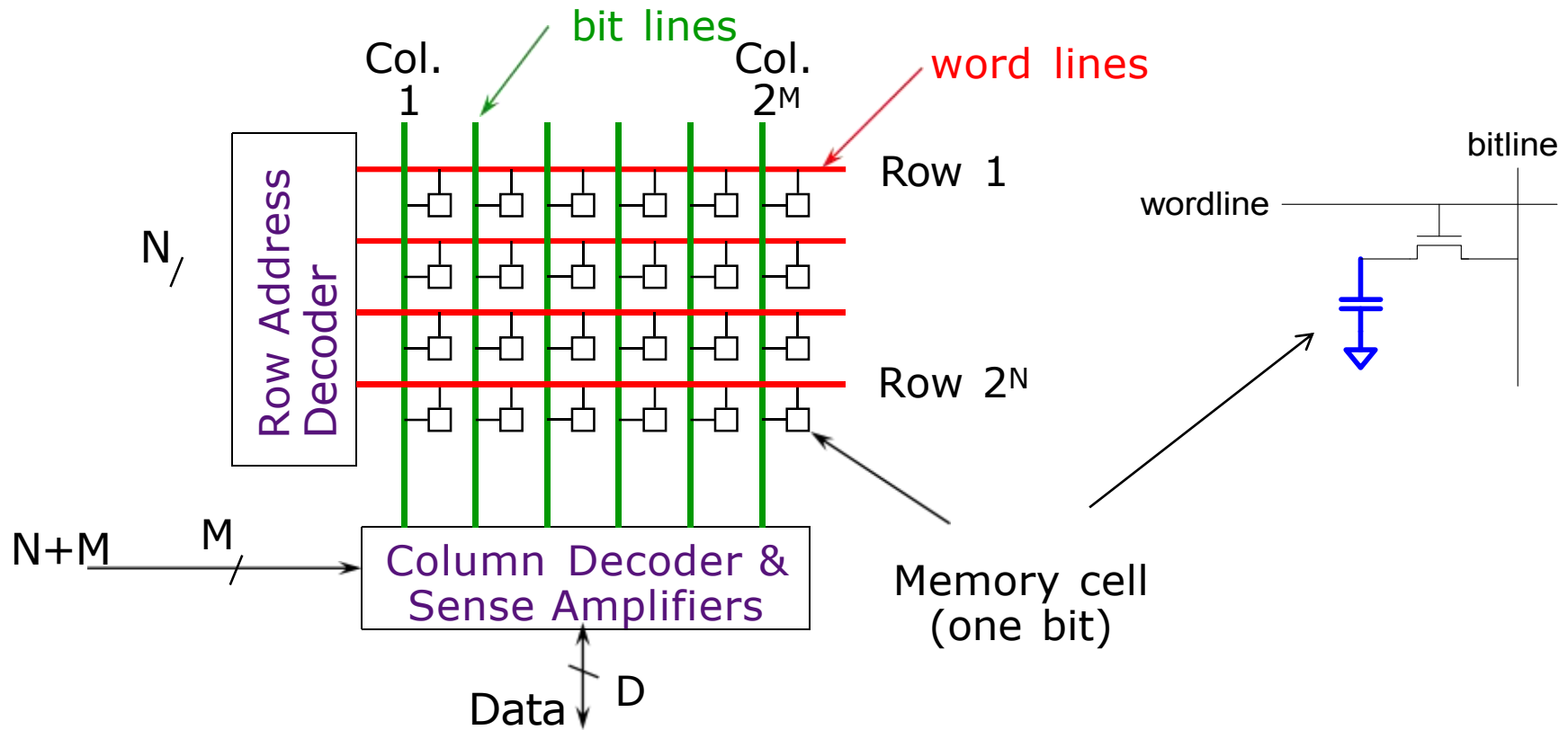
□ 相变存储器(PCM, 3D XPoint)

- 稍慢, 但密度比DRAM高很多，非易失

Memory技术 : DRAM

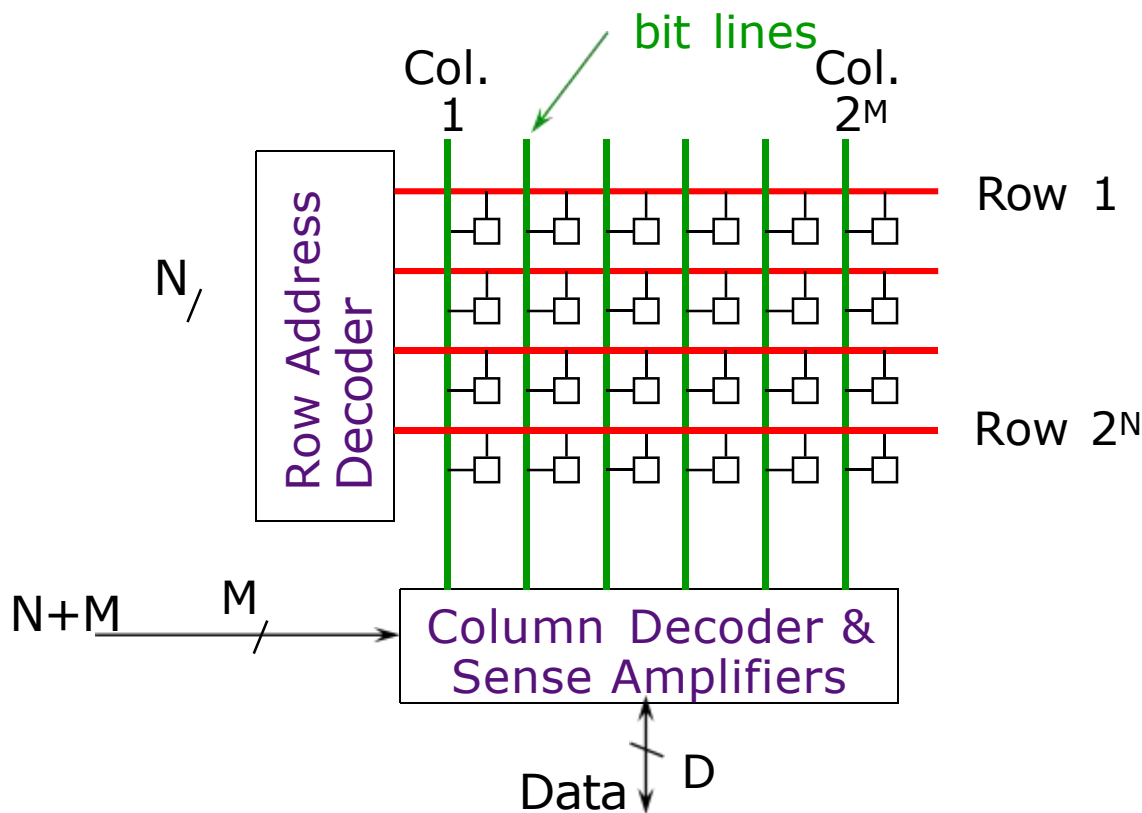


Memory技术 : DRAM



数据存储在2维阵列中

Memory技术 : DRAM

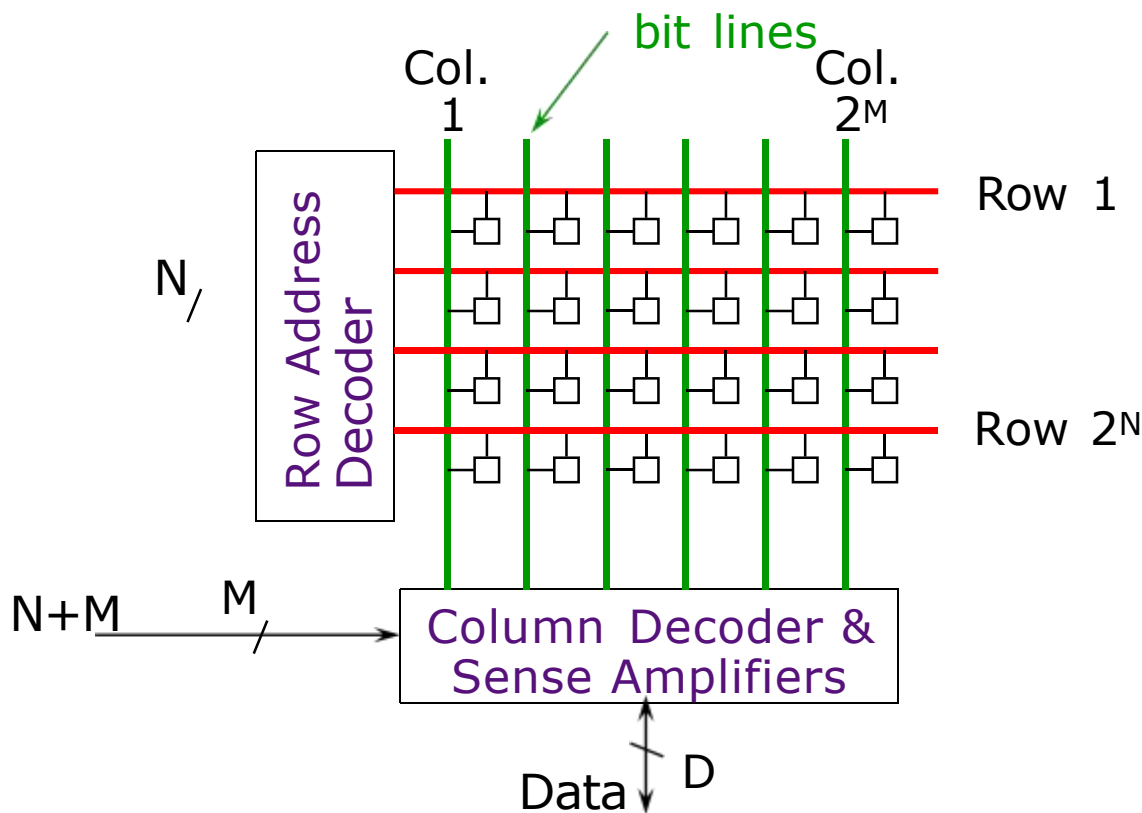


数据存储在2维阵列中

数据访问步骤 :

1. 行地址译码 & 驱动字线
2. 选中的bits驱动位线
 - 读取整行数据
3. 放大一行数据
4. 列地址译码 & 选择整行数据的一个子集
 - 输出数据
5. 位线预充电
 - 准备下次访问

Memory技术 : DRAM



数据访问步骤：

1. 行地址译码 & 驱动字线
2. 选中的bits驱动位线
 - 读取整行数据
3. 放大一行数据
4. 列地址译码 & 选择整行数据的一个子集
 - 输出数据
5. 位线预充电
 - 准备下次访问

破坏性读取

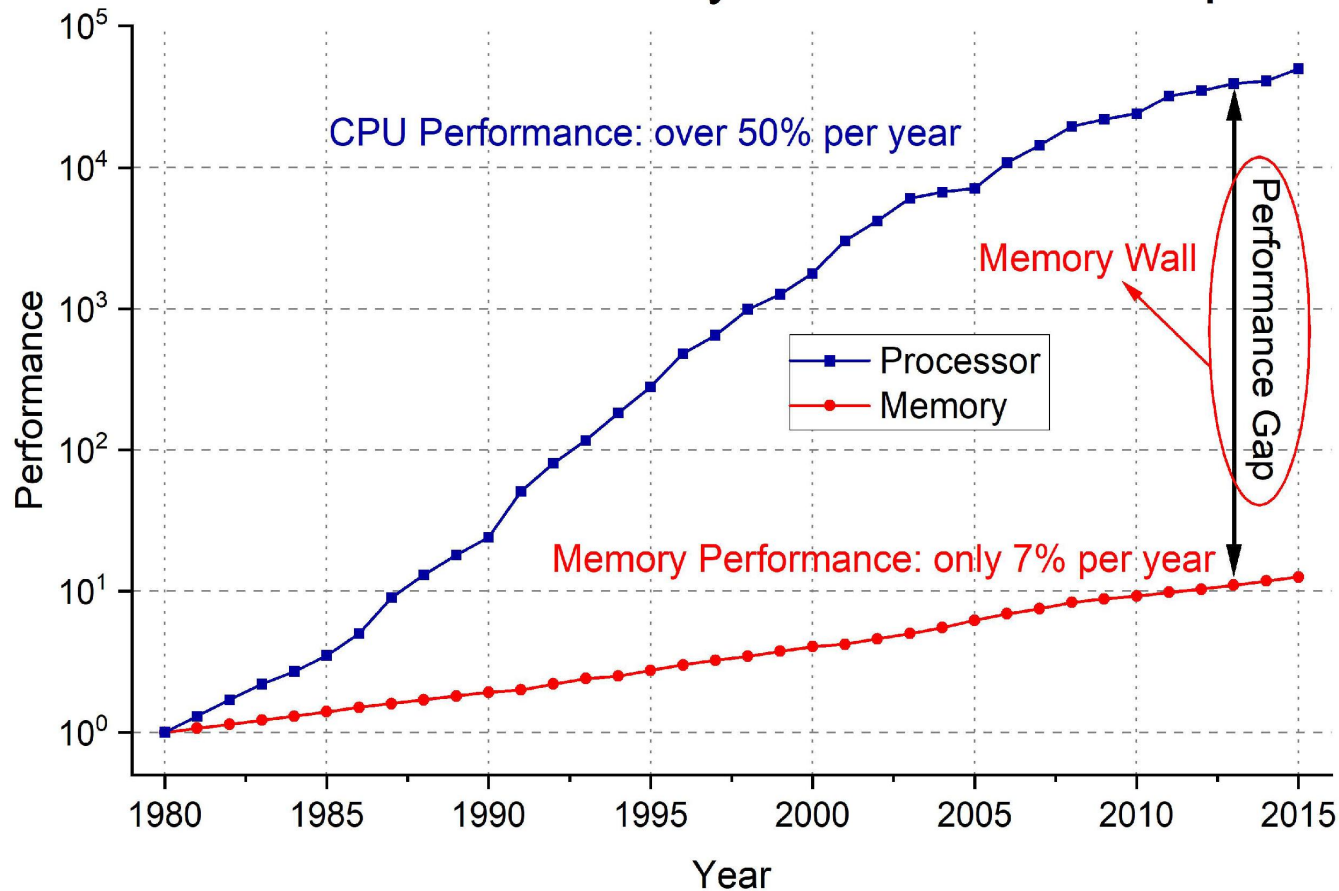
电容会随着时间漏电

刷新: 一个DRAM控制器必须定期读取每一行, 以确保在允许的刷新时间 (几十毫秒) 内恢复电荷

构建大容量Memory

- **问题:** 如果用单个存储阵列，随着容量增大访问延迟会增加，而且还不能并行访问
- **目标:** 降低访问延迟，支持并行访问
- **思路:** 将单个存储阵列划分成多个能够并行访问的banks
 - 每个bank容量比总容量小
 - 访问不同的banks可以并行进行
- **主要问题:** 如何将数据映射到不同的banks?

Processor-DRAM Gap (latency)



Processor和DRAM之间的性能差异越来越大

有没有更快的存储设备?

□ 触发器 (or 锁存器)

- 非常快, 并行访问
- 非常贵 (1 bit需要几十个晶体管)

□ Static RAM (SRAM)

- 比较快, 每次只能读/写一个word
- 贵 (1 bit需要6个晶体管)

□ Dynamic RAM (DRAM)

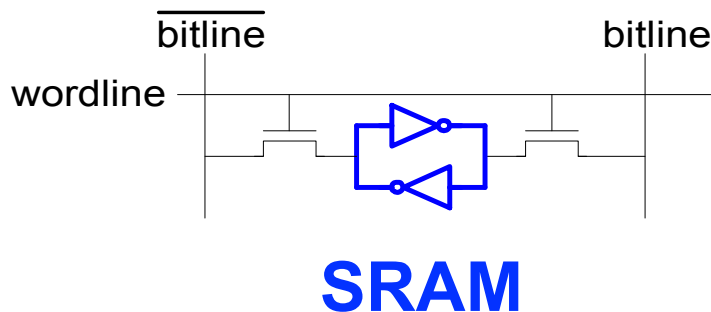
- 慢, 每次读写一个word, 破坏性读 (refresh)
- 便宜 (1 bit 只需要一个晶体管 + 一个电容)

Memory 技术: SRAM

□ **静态随机存取存储器**(Static Random Access Memory)

□ **两个交叉耦合的反相器存储1 bit数据**

- 反馈路径使得存储在“单元”中的值能够持久存在
- 存储：4个晶体管
- 访问：2个晶体管



DRAM vs. SRAM

□ DRAM

- 访问速度慢 (capacitor)
- 密度高 (1bit 1个晶体管)
- 价格低
- 需要刷新 (power, performance, circuitry)

□ SRAM

- 访问速度快 (no capacitor)
- 密度低 (1bit 需要6个晶体管)
- 价格高
- 不需要刷新

理想的内存应该是...

- 零访问时间 (latency)
- 无限大容量
- 零开销
- 无限高带宽 (支持并行访问)

现实情况是...

□ 内存容量越大延迟越高

- 内存容量越大 → 定位到具体数据位置的时间越长

□ 速度越快的设备价格越贵

- SRAM vs. DRAM vs. Disk vs. Tape

□ 带宽越高，代价越高

- 需要更多的banks, 更多的端口, 更高的频率

现实情况是...

□ 内存容量越大延迟越高

- SRAM, 512 Bytes, sub-nanosec
- SRAM, KByte~MByte, ~nanosec
- DRAM, Gigabyte, ~50 nanosec
- Hard Disk, Terabyte, ~10 millisec

□ 速度越快的设备价格越贵

- SRAM, < 10\$ per Megabyte
- DRAM, < 1\$ per Megabyte
- Hard Disk < 1\$ per Gigabyte

内存层次 (Memory Hierarchy)

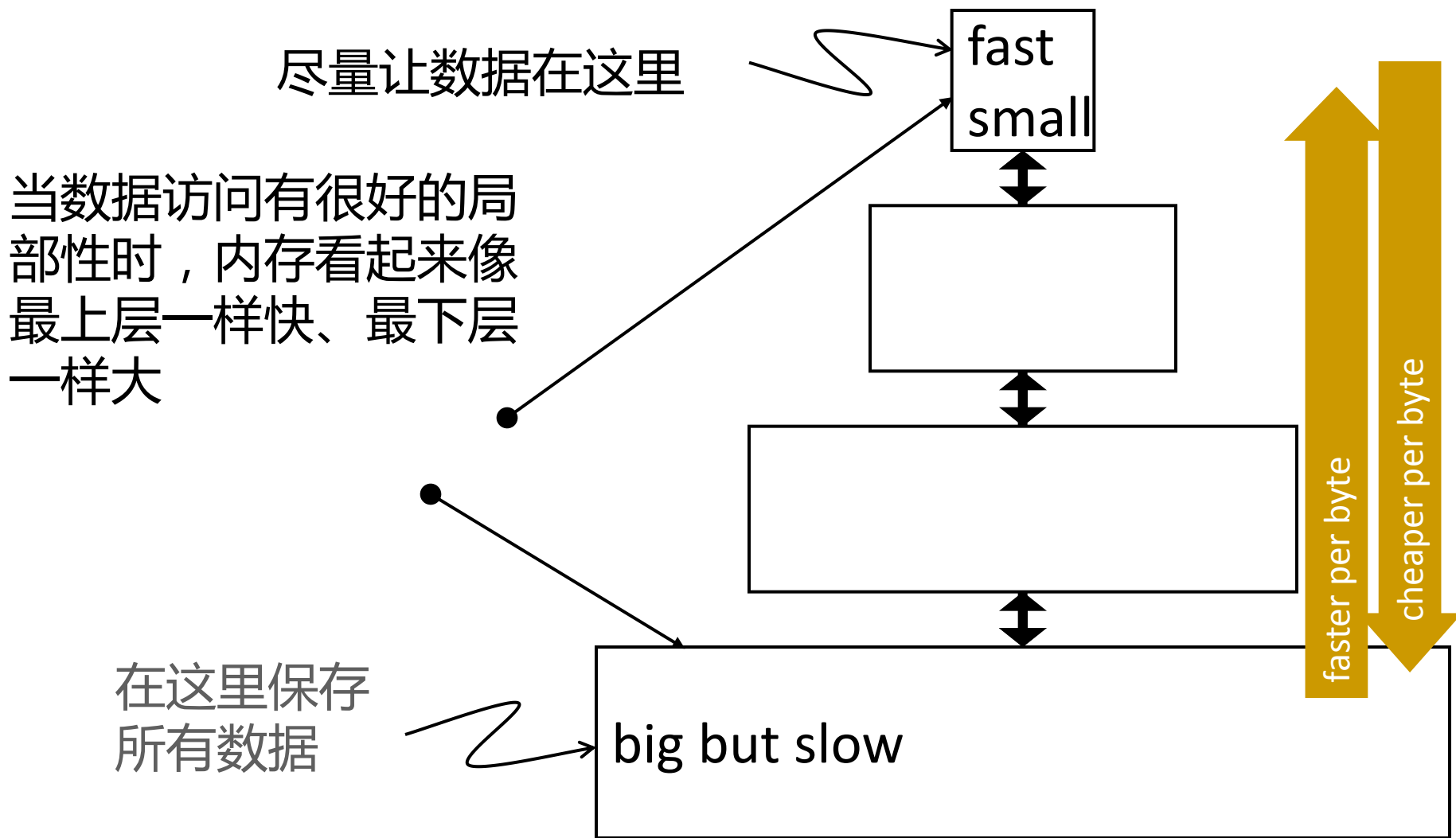
□ **目标**：既快又大的内存

□ 采用单层的存储结构很难实现

□ **思路**：采用多层次存储结构

- 离处理器越远的层级空间越大、速度越慢
- 确保处理器需要的大部分数据都保存在速度较快的层级中

内存层次 (Memory Hierarchy)



内存局部性

- 一个典型程序的内存访问会表现出很多局部性
 - 一般程序都有很多“循环”
- **时间局部性:** 一个程序倾向于在一个小的时间窗口内多次访问相同的内存位置
- **空间局部性:** 一个程序倾向于一次访问一片相近的内存位置
 - 1. 指令内存的访问（指令的地址通常是连续的）
 - 2. 数组访问

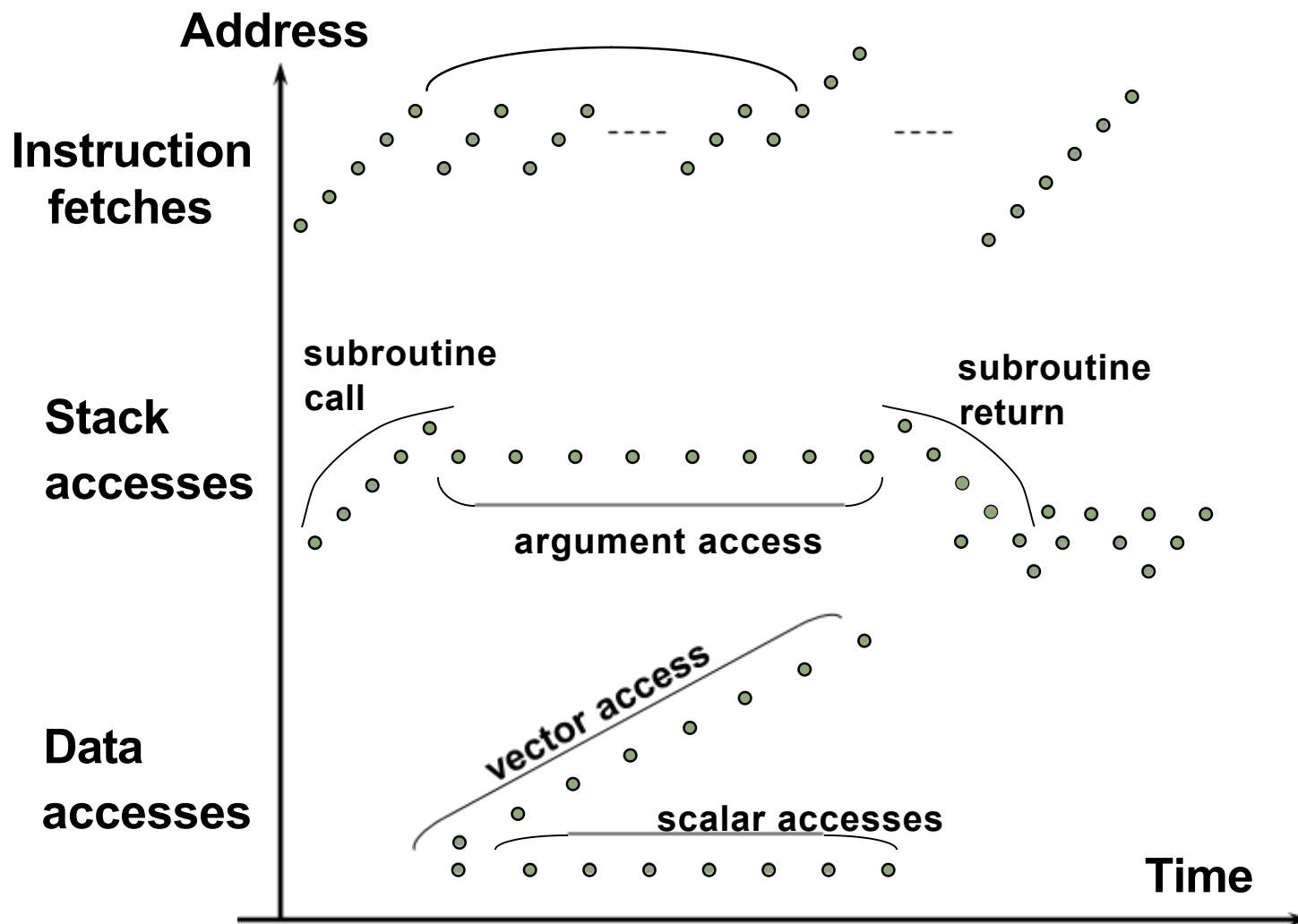
时间和空间局部性

- 哪里表现出空间局部性?
- 哪里表现出时间局部性?

```
int sum = 0;
int X[1000];

for(int c = 0; c < 1000; c++) {
    sum += X[c];
}
```

典型的内存访问模式



Caching: 利用时间局部性

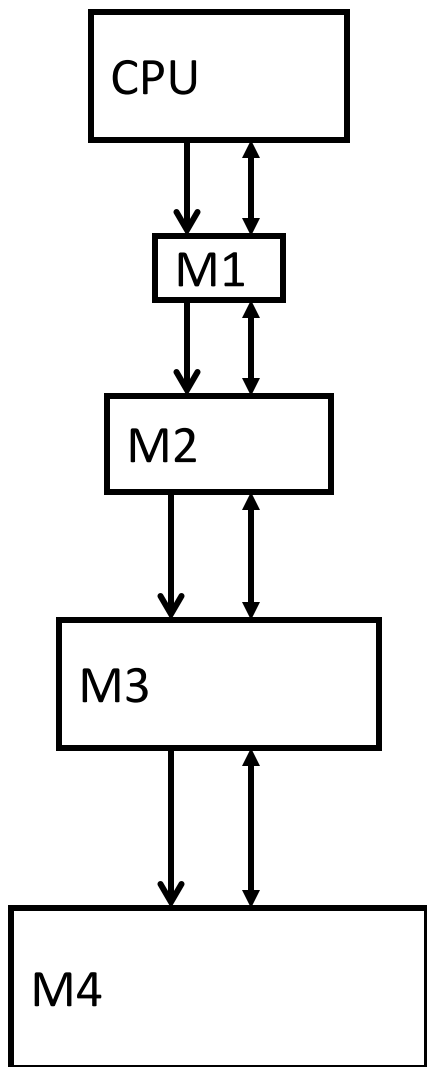
- **思路:** 将最近访问的数据存放在访问速度快的地方(cache)
- **期望:** 数据很快会被再次访问

Caching: 利用空间局部性

- **思路:** 将与近期访问的数据地址相邻的数据放在访问速度快的地方
 - 逻辑上将内存划分成大小相等的块(blocks)
 - 访问某个数据，就将数据所在的块整个缓存

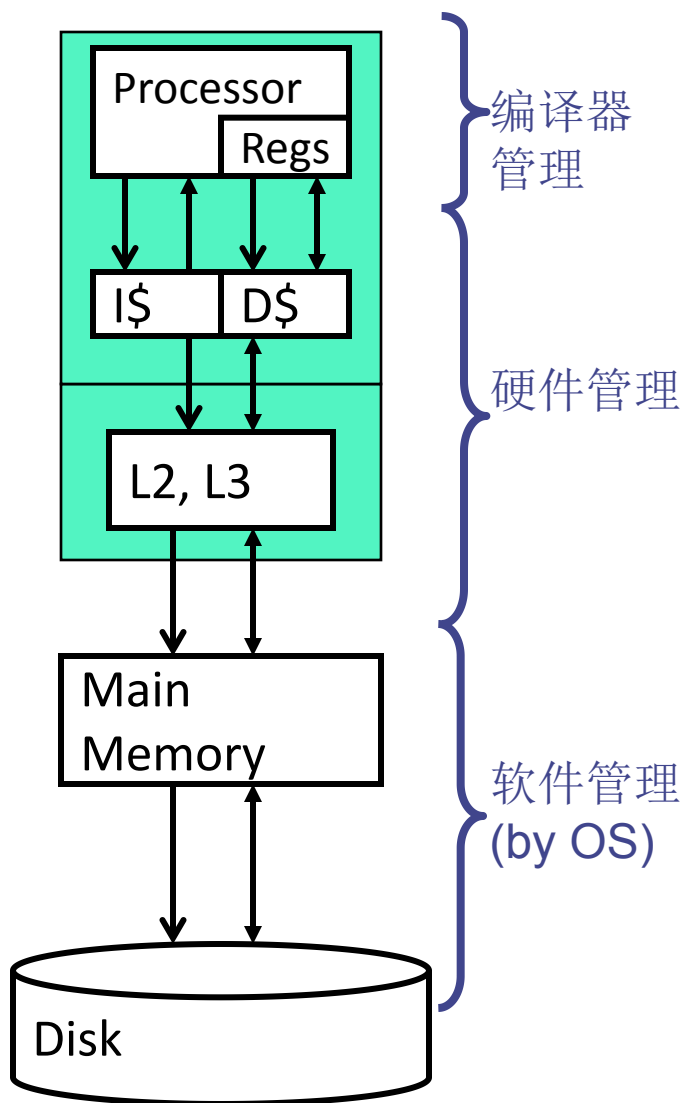
- **期望:** 相邻的数据很快会被访问

利用局部性: Memory Hierarchy



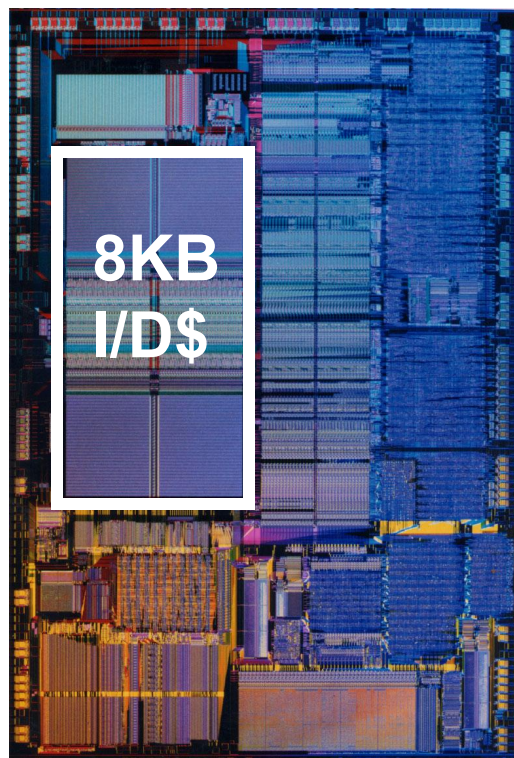
- **层级式的存储结构**
 - 上面的层级
 - Fast ↔ Small ↔ Expensive
 - 下面的层级
 - Slow ↔ Big ↔ Cheap
- **通过总线相连**
 - 同样存在带宽和延迟的问题
- **最经常访问的数据在M1**
 - M1 + 次经常访问的在M2, etc.
- **平均访问时间**
 - $t_{avg} = t_{access} + (\%_{miss} * t_{miss})$

现代处理器的存储层次

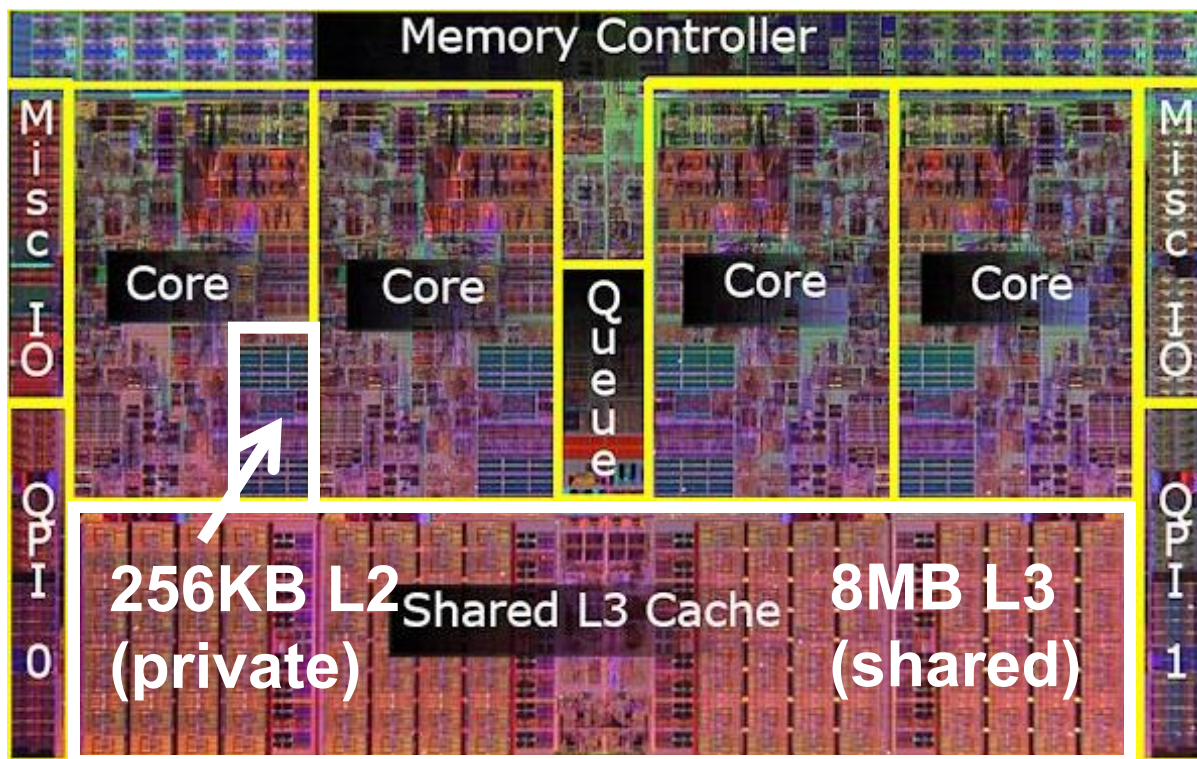


- 0th level: **寄存器**
- 1st level: **L1 Cache**
 - 指令Cache (I\$) 和数据Cache (D\$)分开
 - 大小为 8KB~64KB
- 2nd level: **L2 and L3 cache**
 - 片上, 通常采用 SRAM
 - L2 Cache大小为 256KB~512KB
 - L3 (Last level cache) 大小为4MB~16MB
- 3rd level: **main memory**
 - DRAM (“Dynamic” RAM)
 - 大小为 1GB~4GB(个人电脑)
 - 服务器的内存可能超过100GB
- 4th level: **磁盘**
 - 采用硬盘或者闪存等

存储层次的演变



Intel 486



Intel Core i7 (quad core)

- 现代处理器芯片Cache面积占 30–70%

Cache

Cache 基本概念

□ Block (or Cache Line): Cache中的最小数据单元

- Cache被划分为若干个blocks，每个block可以存储从主存中复制过来的一部分数据
- 主存在逻辑上也被划分为blocks，每个block映射到cache中的一个位置
- 数据以block为单位进行cache相关操作

□ 内存访问时:

- **HIT**: 如果在cache中, 使用cache中的数据(不必再访问内存)
- **MISS**: 如果不在cache中, 将数据从内存复制到cache

□ Cache设计中的一些重要决策：

- **放置(Placement)**: 将block放在cache的哪个位置?
- **替换(Replacement)**: 为了腾出空间，移除哪些数据?
- **写策略(Write policy)**: 如何处理写请求?

Cache寻址

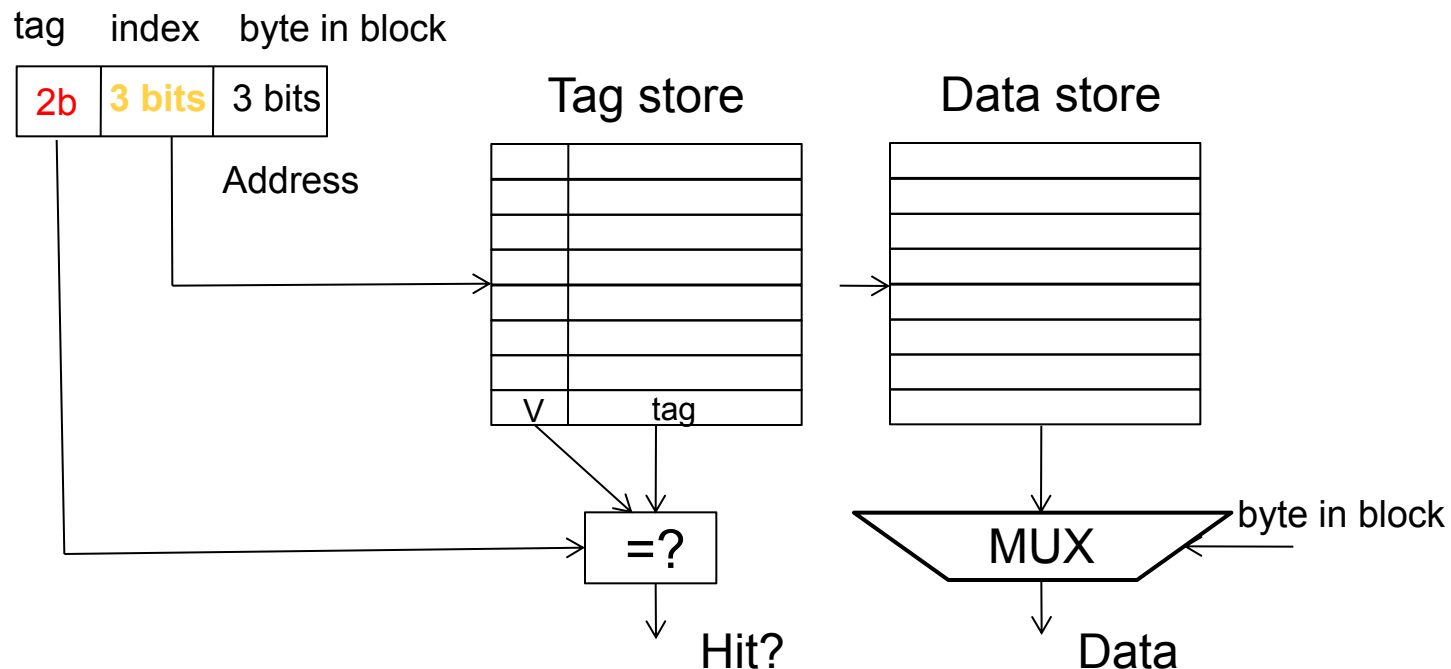
- 内存逻辑上被分成固定大小的blocks
- 每个block映射到cache中的一个位置, 由映射算法根据内存地址中的index bits来决定
- **Cache 访问:**
 - 1) 先根据内存地址中的index bits索引到指定的cache block(包含tag和data)
 - 2) 然后检查tag中的valid bit
 - 3) 最后将内存地址中的tag bits和cache中的tag进行比较
- 如果一个block在cache中(cache hit), tag中的valid bit应该置位, 并且tag应该与内存地址中的tag bits相同

放置策略：直接映射(Direct-Mapped)

Block: 00000
Block: 00001
Block: 00010
Block: 00011
Block: 00100
Block: 00101
Block: 00110
Block: 00111
Block: 01000
Block: 01001
Block: 01010
Block: 01011
Block: 01100
Block: 01101
Block: 01110
Block: 01111
Block: 10000
Block: 10001
Block: 10010
Block: 10011
Block: 10100
Block: 10101
Block: 10110
Block: 10111
Block: 11000
Block: 11001
Block: 11010
Block: 11011
Block: 11100
Block: 11101
Block: 11110
Block: 11111

Main memory

- 假设内存是字节寻址: 大小为256 bytes, block 大小为8 bytes , 共分为32个blocks
- 假设Cache: 大小为64 bytes, 包含8个blocks
- 直接映射: 一个block只能映射到一个位置



Index相同的内存地址竞争相同的位置

- 导致冲突未命中(conflict misses)

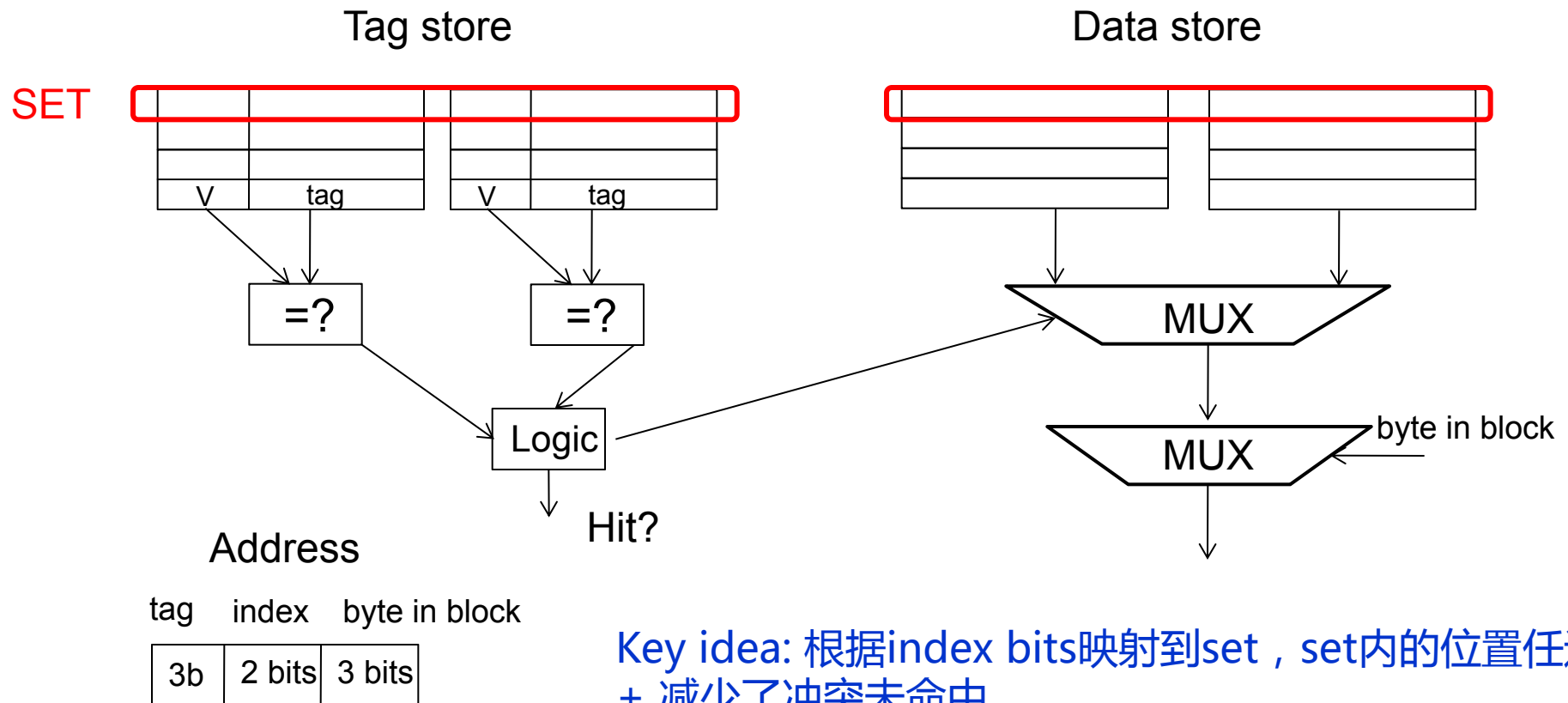
直接映射的 Cache

- **直接映射的 cache:** 内存中index相同的两个blocks不能同时在cache中出现
 - One index $\rightarrow$ one entry

- **如果交替访问index相同的两个blocks , 会导致0%命中率**
 - 假设地址 A and B 有相同的index bits(但tag bits不同)
 - A, B, A, B, A, B, A, B, ... $\rightarrow$ conflict in the cache index
 - 所有访问都是未命中

放置策略：组相联(Set Associativity)

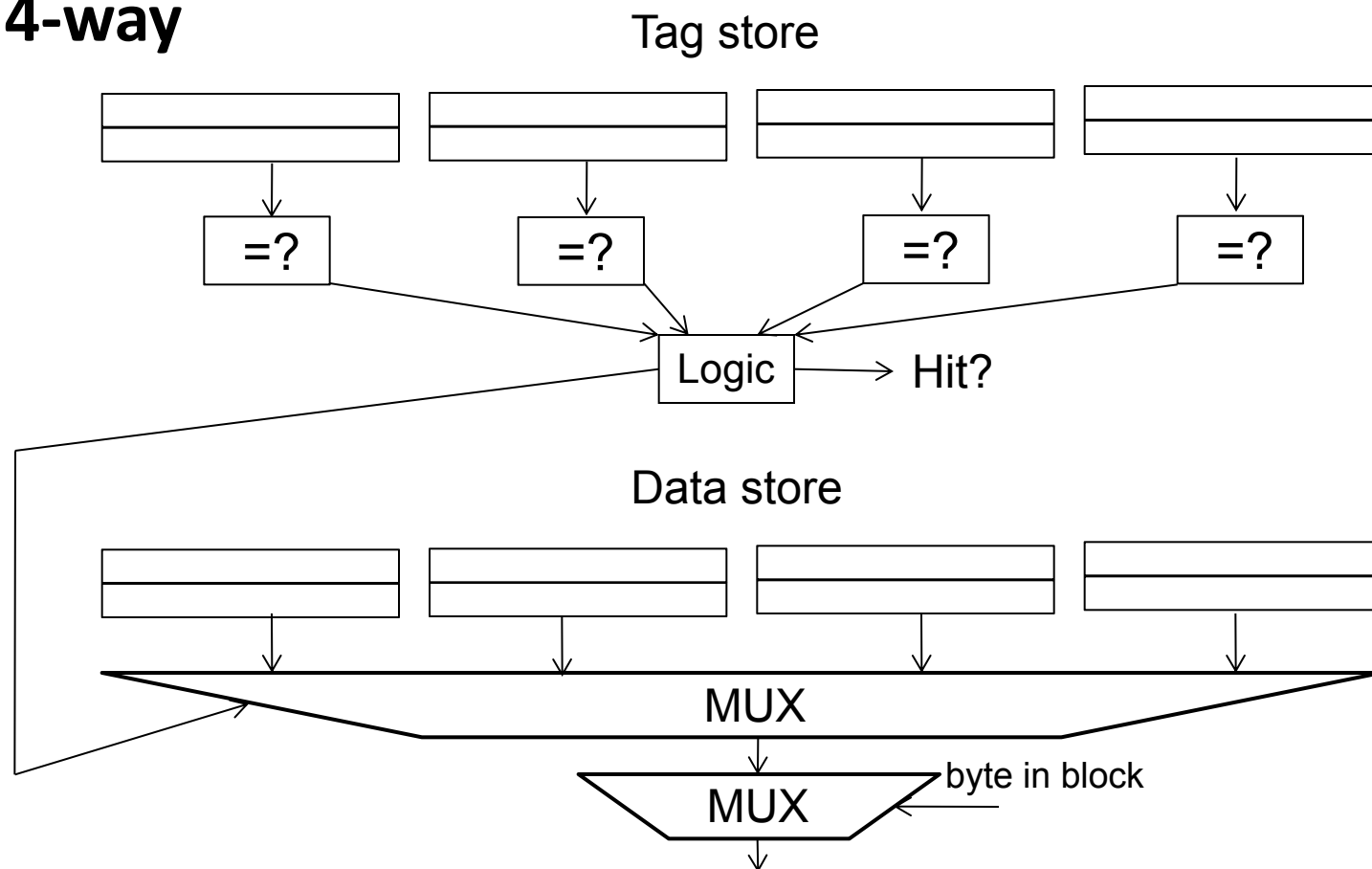
- ❑ 在直接映射中，地址 0 and 8 总是冲突
- ❑ 不是只有1列(包含8个blocks)，而是有2列(每列包含4个blocks)



Key idea: 根据index bits映射到set，set内的位置任选
+ 减少了冲突未命中
-- 更复杂, 访问更慢, tag更长(index bits变短了)

更高关联度

4-way



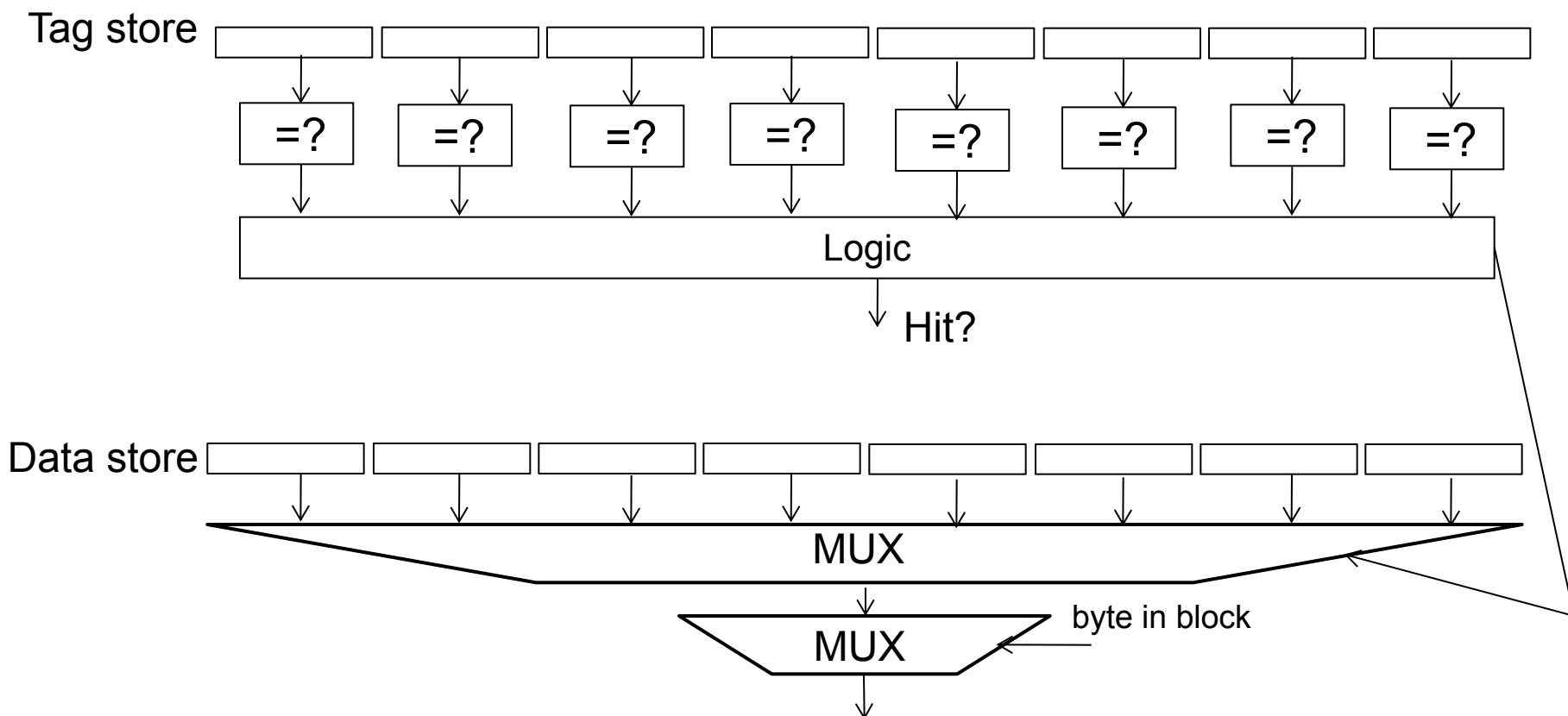
+ 冲突未命中更少了

-- tag比较器增加，data mux变宽，tags更长

放置策略：全相联(Full Associativity)

□ 全相联

- 1个block可以放在cache的任何位置



Cache主要参数的影响：Capacity

□ Cache size: 总容量

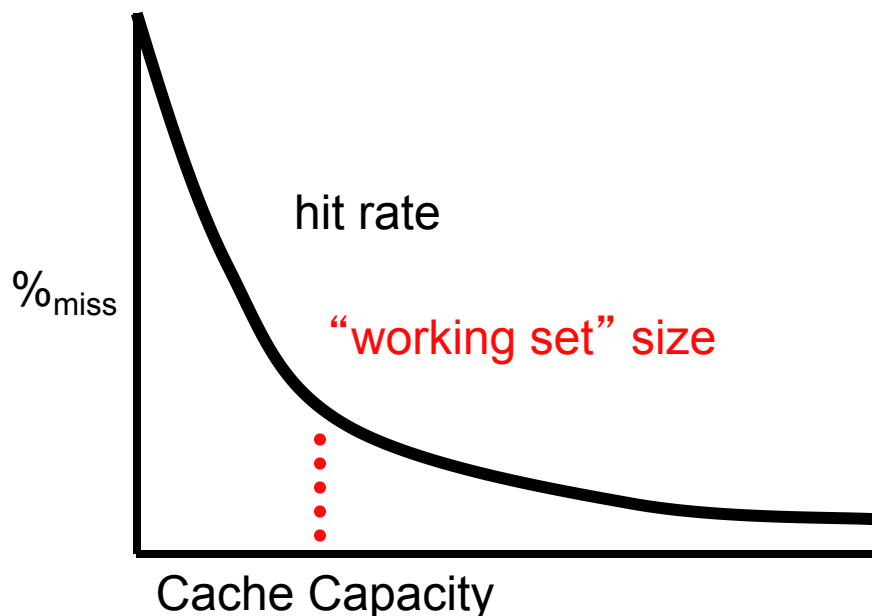
- 更大的缓存可以更好地利用时间局部性

□ Cache增大会增加访问延迟

- 越小越快 => 越大越慢
- 访问延迟可能会增加关键路径的延迟

□ Cache太小

- 无法有效利用时间局部性
- 有用的数据频繁被替换



Cache主要参数的影响：关联度

□ **关联度**: 每个组(set)里能放几个blocks

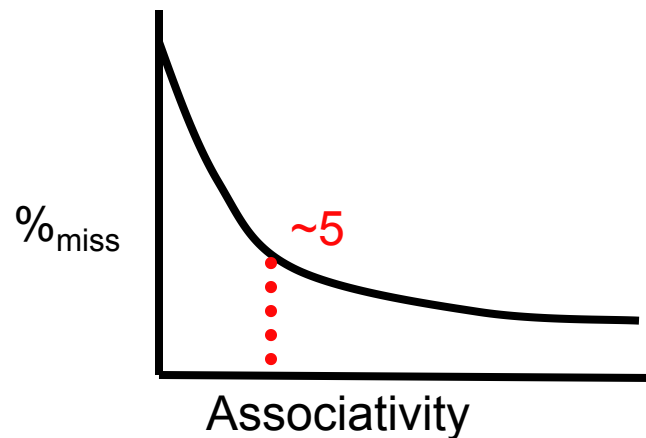
□ **关联度更高**

++ 命中率越高

-- 访问时间更长

-- 硬件更复杂 (更多比较器)

□ **随着关联度增加，命中率增长趋缓**



Cache主要参数的影响：Block Size

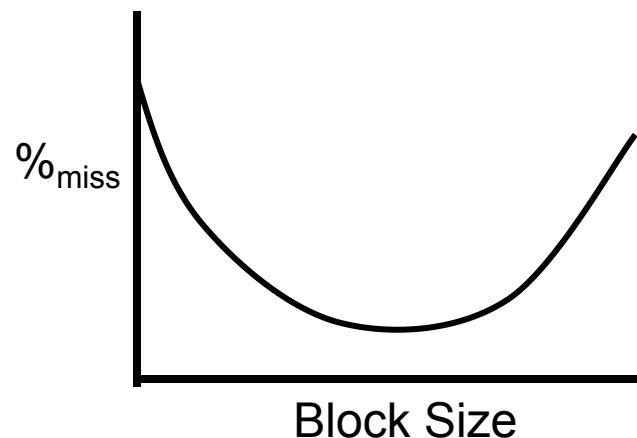
□ 增大block size有正反两方面影响

+ 空间预取 (正面)

- 地址相邻的blocks被提前预取
- 有利于降低miss rate

- 产生干扰 (反面)

- 能放入cache的blocks变少了
- 可能增加miss rate
- 特殊情况：全局只有1个block



□ 两方面影响同时存在

- 具体哪个影响大取决于workload

替换策略(Replacement Policy)

□ Cache miss发生时，set中的哪个block将被替换？

- 优先替换invalid block (还没放置任何数据)
- 如果都是valid, 由替换策略决定
 - ✓ **Random**
 - ✓ **FIFO (first-in first-out)**
 - ✓ **LRU (least recently used)**
 - 适合空间局部性 (未来最不可能用到的block)
 - ✓ **NMRU (not most recently used)**
 - LRU的近似策略，更容易硬件实现
 - 2路组相联(2-way set assoc.)时就是LRU
 - ✓ **Belady's**: 替换最晚用到的block
 - 最优策略，但无法实现

硬件实现 LRU

- **思路:** 替换最近最少使用的block
- **问题:** 需要记录set内所有block的访问顺序

- **2路组相联的cache:**
 - 需要怎么做来实现精确的LRU算法?

- **4路组相联的cache:**
 - 需要怎么做来实现精确的LRU算法?
 - 一个set中有4个blocks, 一共有多少种可能的顺序?
 - 需要多少bits来记录一个block的顺序?
 - 需要什么样的逻辑来决定哪个block被替换?

LRU的近似策略

- 大多数现代处理器在cache相联度较高时不实现精确的LRU策略
- 为什么?
 - 精确 LRU 硬件实现复杂
 - LRU 也是一种近似算法(非最优算法)
- 例如:
 - Not MRU (not most recently used)
 - Hierarchical LRU: 将每个set里面的blocks分成M组, 以组为单位进行LRU

替换策略: LRU or Random

□ LRU vs. Random: 哪个更好?

- 例如: 4-way cache, 循环访问 A, B, C, D, E
 - ✓ LRU的命中率为 0%

□ Set thrashing(抖动): 当数据集大小大于关联度时

- Random 替换策略此时表现更好

□ 实际上:

- 取决于工作负载
- LRU 和 Random 的命中率相似

最优替换策略

□ Belady's 算法

- 替换最晚访问的block
- Belady, "A study of replacement algorithms for a virtual-storage computer," IBM Systems Journal, 1966.
- 如何实现?

□ 是否可以最小化 miss rate?

□ 是否可以最小化访问延迟?

- 不能. 每个block的cache miss延迟不一样!
- 两个原因: 远程cache和本地cache的miss代价不同, 多个miss可能并行处理(开销平摊)
- Qureshi et al. "A Case for MLP-Aware Cache Replacement," ISCA 2006.

Cache 写策略

□ 目前我们一直考虑读操作

- Instruction fetches, loads

□ Cache写操作呢?

□ 一些新问题

- Tag/data 访问顺序
- 写穿透(Write-through) vs. 写返回(write-back)
- 写分配(Write-allocate) vs. 写不分配(write-not-allocate)
- 掩盖写延迟

Tag/Data 访问顺序

□ Cache读: 读tag和读数据可以并行进行

- 如果Tag最终没有match, 丢弃数据

□ Cache写: 读tag和写数据能否并行? No!

- 如果tag没有match, 数据就写错了
- 如果每个set里面有多个blocks, 写哪一个也不确定

□ Cache写是一个两阶段操作

- Step 1: 读tag进行对比
- Step 2: match之后再写cache

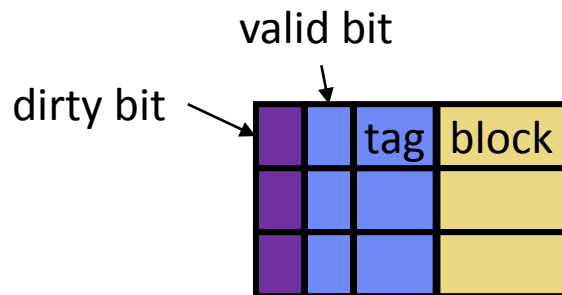
写操作如何传递 (Cache Hit)

- 何时将新数据传递到更下层的cache或memory?
- **Option #1: 写穿透(Write-through): 立即传递**
 - Cache命中, 更新cache
 - 立即向下一层写
- **Option #2: 写返回(Write-back): 当block被替换时**
 - 同一个block在不同层的cache和memory中有不同的版本!
 - 每个block需要一个额外的 “**dirty**” 位
 - 替换 **clean** block: 不需要额外操作
 - 这个block只有1个版本
 - 替换 **dirty** block: 需要向下一层cache写
 - 这个 dirty block 是最新版本

写返回(Write-back)操作

□ 每个cache block有一个dirty bit

- 有两种状态：**clean** or **dirty**



initial state

-	I	-	-
---	---	---	---

after `ld r0 <= [A]`

C	V	A	0x01
---	---	---	------

after `st r1 => [A]`

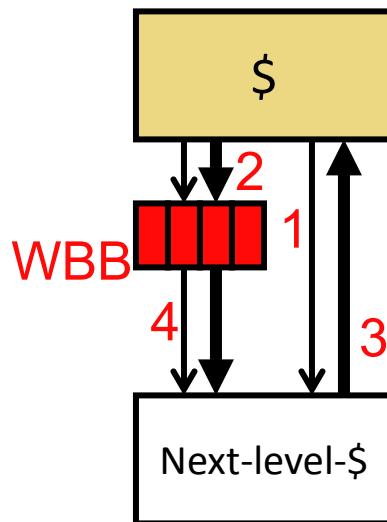
D	V	A	0x02
---	---	---	------

写返回(Write-back)优化

□ Writeback-buffer (WBB)

- **场景**：当上一级Cache的dirty block被替换出去，要写入下一级Cache，但该block不在下一级Cache中
- **Step#1**: 给下一级cache发送“填充”请求
- **Step#2**: 在等待下一级cache填充的过程中, 将dirty block放入buffer
- **Step#3**: 下一级cache填充完毕
- **Step#4**: 将dirty block从buffer中取出并写入下一级cache

减少等待延迟！



Cache写策略对比

□ 写穿透(Write-through)

- 需要额外带宽
 - 考虑反复写命中的情况
- 下层cache需要处理小的写请求 (1, 2, 4, 8-bytes)
- + 不需要dirty bits
- + 不需要处理 “写返回” 操作
 - 常用在GPUs, 因为GPU程序空间局部性低

□ 写返回(Write-back)

- + 优点: 带宽使用少
 - 和 “写穿透” 的优缺点相反
 - 常用在CPU中

处理Write Miss

- **写分配(Write-allocate):** 先从下一层cache将数据读上来，然后再写
 - + 减少了读操作的misses (下一次读该block就能命中)
 - 需要额外的带宽
 - 很常用 (尤其是和 “写返回” 搭配使用)

- **写不分配(Write-non-allocate):** 直接往下一层写, 不将数据读入本层cache
 - 潜在地增加了读操作的misses
 - + 使用较少的带宽
 - 通常和 “写穿透” 搭配使用

Write Miss 和 Store Buffer

□ 发生 read miss 时

- 流水线必须停顿

□ 发生 write miss 时

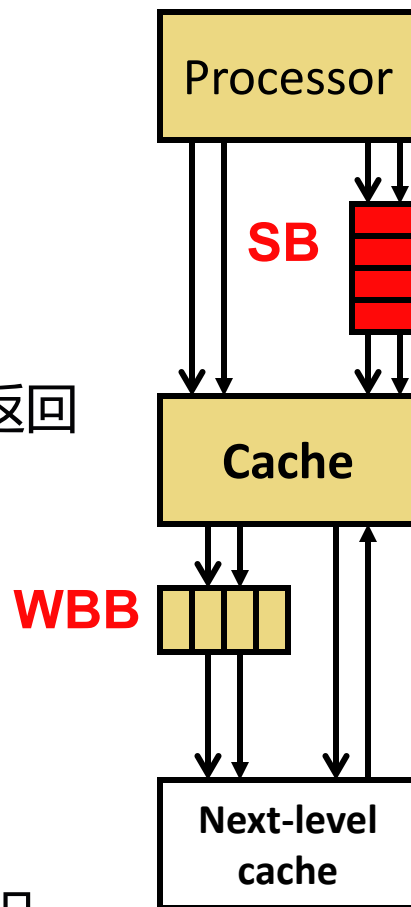
- 没有指令在等数据，为何要停顿？

□ Stall Buffer

- Store指令将address/value放入store buffer, 然后返回
- Store buffer在后台将数据写入cache
- Loads需要检查store buffer
- 消除了write miss时的停顿

□ Store buffer vs. writeback-buffer

- Store buffer在cache之前, 掩盖store miss延迟
- Writeback buffer在cache之后, 掩盖writeback延迟



提升Cache性能

提升Cache性能

程序执行时间 = 指令数 * CPI * 时钟周期

$\text{CPI (with cache)} = \text{CPI_base} + \text{CPI_cachepenalty}$

$\text{CPI_cachepenalty} = \dots\dots\dots$

1. 降低 miss rate
2. 降低 miss penalty
3. 降低 hit latency

提升Cache性能

程序执行时间 = 指令数 * CPI * 时钟周期

CPI (with cache) = CPI_base + CPI_cache_penalty

CPI_cache_penalty =

1. 降低 miss rate
2. 降低 miss penalty
3. 降低 hit latency

Cache Miss 的三种情况 (3C)

❑ Compulsory miss

- 第一次访问一个block肯定会发生miss
- 后续对该block的访问应该命中，除非block因为下面的原因被替换

❑ Capacity miss

- cache空间不足以缓存所有的数据（小于内存）
- 即使采用大小相等的全相联cache、以及最优的替换算法，仍然会发生的那些miss

❑ Conflict miss

- 既不是compulsory miss，也不是capacity miss的那些miss

1. 增大Cache Size

- 可以降低conflict miss和capacity miss

	增大cache size	增加关联度	增加block size
Compulsory misses	=		
Capacity misses	↓		
Conflict misses	↓		
Hit latency	↑		
Miss latency	=		

缺点：

- 访问延迟增加
- 能耗更高

2. 增加关联度

- 可以降低conflict miss

	增大cache size	增加关联度	增加block size
Compulsory misses	=	=	
Capacity misses	↓	=	
Conflict misses	↓	↓	
Hit latency	↑	↑	
Miss latency	=	=	

缺点：

- 访问延迟增加

3. 增加block size?

- 可以降低compulsory miss

	增大cache size	增加关联度	增加block size
Compulsory misses	=	=	↓
Capacity misses	↓	=	↓
Conflict misses	↓	↓	?
Hit latency	↑	↑	=
Miss latency	=	=	↑

缺点：

- 如果访存空间局部性不高，会浪费cache空间

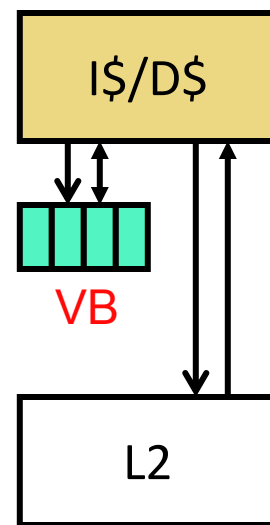
4. Victim Buffer

□ Conflict misses: 关联度不够

- 增加关联度成本太高

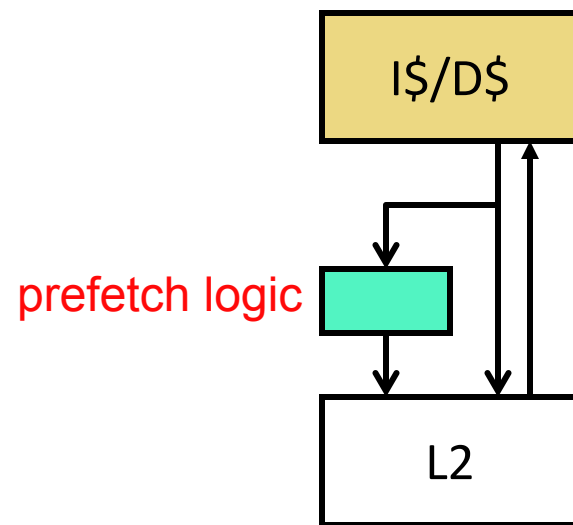
□ Victim buffer (VB): 小的全相联cache

- 在指令/数据cache miss的路径上
- 小 (e.g., 8 entries) 所以非常快
- 从指令/数据cache淘汰的blocks放入VB
- 如果发生miss, 检查VB: hit? 将block放回指令/数据cache
- + 非常高效



5. 预取 (Prefetching)

- 提前将数据放入cache
- **关键:** 能够预测将要访问哪些blocks
 - 可以基于硬件或软件实现
- 简单硬件预取：next block prefetching
 - 对于顺序执行的指令效果好
 - 对于数组效果好
- 表格驱动的硬件预取
 - 采用预测器探测内存访问模式
- 预取的有效性取决于
 - Timeliness: 预取足够提前
 - Coverage: 尽可能覆盖更多的miss
 - Accuracy: 不会预取无关数据



6. 软件方法 - 重构数据访问模式

□ 循环交换: 空间局部性

- **Poor code:** `X[i][j]` followed by `X[i+1][j]`

```
for (j = 0; j<NCOLS; j++)  
    for (i = 0; i<NROWS; i++)  
        sum += X[i][j];
```
- **Better code:**

```
for (i = 0; i<NROWS; i++)  
    for (j = 0; j<NCOLS; j++)  
        sum += X[i][j];
```

6. 软件方法 - 重构数据访问模式

□ 循环分块: 时间局部性

- **Poor code**

```
for (k=0; k<NUM_ITERATIONS; k++)  
    for (i=0; i<NUM_ELEMS; i++)  
        X[i] = f(X[i]);    // say
```

- **Better code**

- ✓ 将数组按照CACHE_SIZE大小分成块
- ✓ 先将一个块的所有相关操作都做完, 再处理下一个块

```
for (i=0; i<NUM_ELEMS; i+=CACHE_SIZE)  
    for (k=0; k<NUM_ITERATIONS; k++)  
        for (j=0; j<CACHE_SIZE; j++)  
            X[i+j] = f(X[i+j]);
```

- 假设知道 CACHE_SIZE, 如何知道?

7. 软件方法 - 重构数据布局

```
struct Node {  
    struct Node* next;  
    int key;  
    char [256] name;  
    char [256] school;  
}  
  
while (node) {  
    if (node→key == input-key) {  
        // access other fields of node  
    }  
    node = node→next;  
}
```

- 基于指针的遍历 (e.g., 链表)
- 假设链表长度很长，每个节点都有唯一的key
- **为何左边的代码对cache不友好?**
 - 节点的“其他成员”（除了指针外）占了很大空间，但很少被访问（if条件很少满足）！

7. 软件方法 - 重构数据布局

```
struct Node {  
    struct Node* next;  
    int key;  
    struct Node-data* node-data;  
}
```

```
struct Node-data {  
    char [256] name;  
    char [256] school;  
}
```

```
while (node) {  
    if (node→key == input-key) {  
        // access node→node-data  
    }  
    node = node→next;  
}
```

□ **思路:** 将频繁访问的数据成员分离出来，单独组成一个数据结构

□ **谁来做这件事?**

- Programmer
- Compiler
 - Profiling vs. dynamic
- Hardware?
- **谁能知道哪些是频繁访问的?**

提升Cache性能

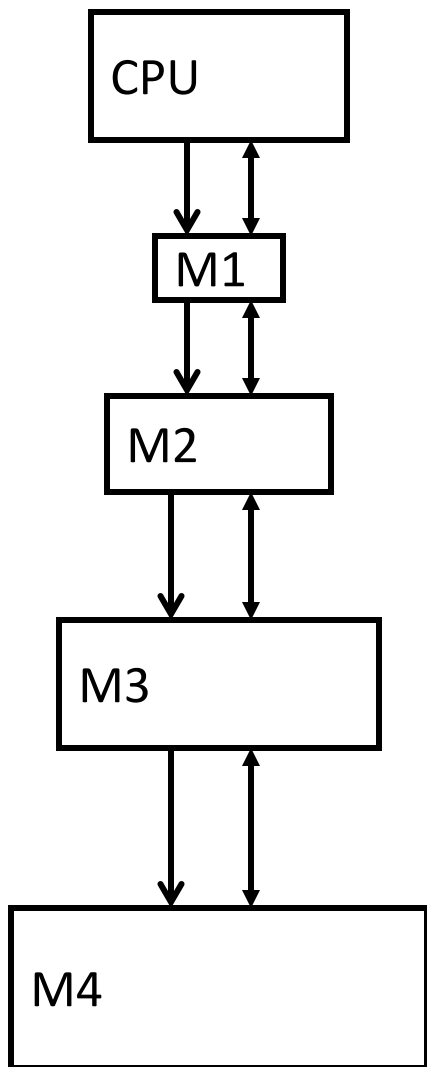
程序执行时间 = 指令数 * CPI * 时钟周期

CPI (with cache) = CPI_base + CPI_cache_penalty

CPI_cache_penalty =

1. 降低 miss rate
2. 降低 miss penalty
3. 降低 hit latency

1. 采用多级 Cache



➤ 层级式的存储结构

➤ 最经常访问的数据在M1

- M1 + 次经常访问的在M2, etc.

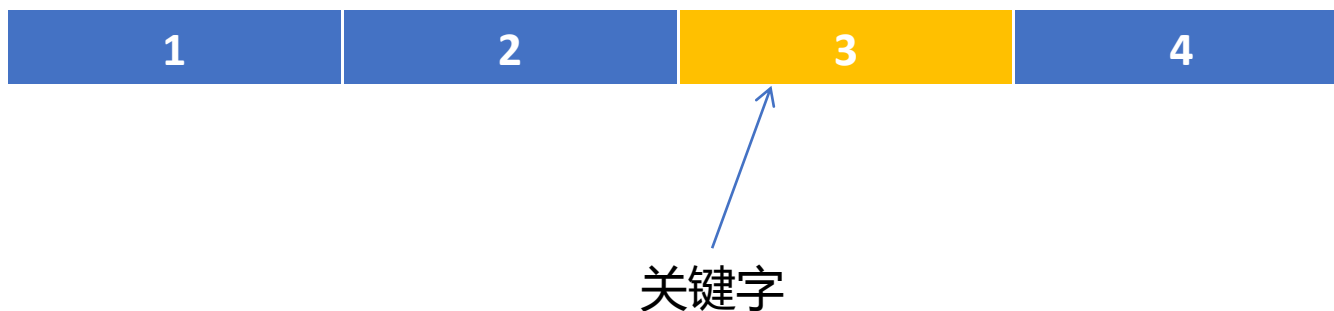
➤ 平均访问时间

- $t_{avg} = t_{access} + (\%_{miss} * t_{miss})$

M2的平均访问时间就是M1的miss penalty !

2. 关键字优先 (Critical Word First)

- ❑ 发生cache miss时，首先识别出请求所在的缓存行中的关键字(word)
- ❑ 处理器优先读取关键字，而不是整个cache line
 - 尽快满足当前指令的数据需求，从而减少访存延迟
- ❑ 处理器并行读取整个cache line
 - 并行传输能够利用总线带宽和处理器与内存之间的带宽



提升Cache性能

程序执行时间 = 指令数 * CPI * 时钟周期

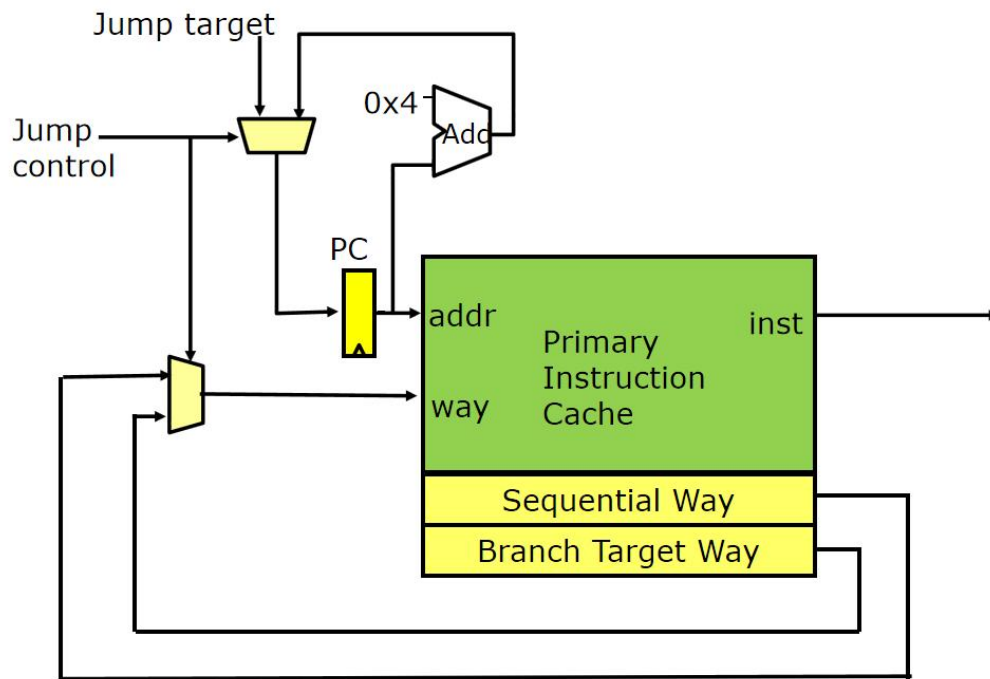
CPI (with cache) = CPI_base + CPI_cache_penalty

CPI_cache_penalty =

1. 降低 miss rate
2. 降低 miss penalty
3. 降低 hit latency

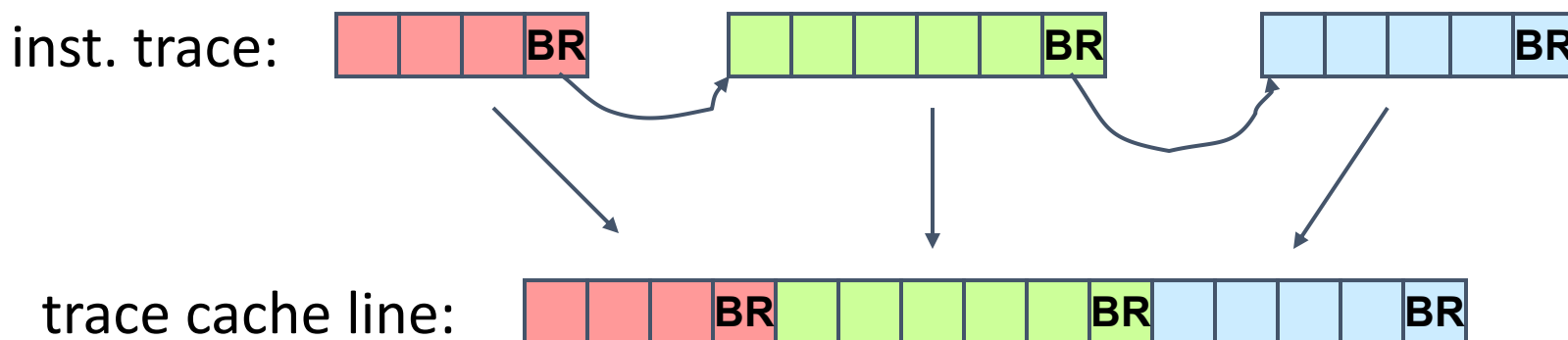
1. 路预测 (Way Prediction)

- 适用于多路组相联Cache
- 每一个set中维护一个预测器, 根据历史信息预测下一次cache会访问哪一路 (block)
 - cache访问先和预测结果比较, 如果正确, 直接返回该block
 - 只比较一次, 节省时间和能耗
- 准确率 $\approx 85\%$



2. Trace Cache

- 适用于指令Cache
- 在执行指令时，处理器通过硬件跟踪单元记录指令执行路径
- 当执行完一个追踪时，将指令trace的多个blocks打包成长trace cache line
- 当处理器需要执行一个新的指令时，首先检查trace cache
 - 如果存在，处理器可以从trace cache中直接获取整个追踪，而无需单独解析和预测每条指令



提高Cache性能的其他方法

□ 降低 miss rate

- 间接增加关联度
 - ✓ hashing, pseudo-associativity, skewed associativity
- 更好的替换策略
- 软件方法

□ 降低 miss latency/cost

- Subblocking/sectoring(将一个block划分为多个subblocks)
- 更好的替换策略
- 非阻塞caches (多个cache misses并行处理)
- 每周期多次访问(降低访问延迟)
- 软件方法

存储层次设计

□ 对于每一层，都需要权衡: t_{access} (命中延迟) vs. $\%_{\text{miss}}$ (未命中率)

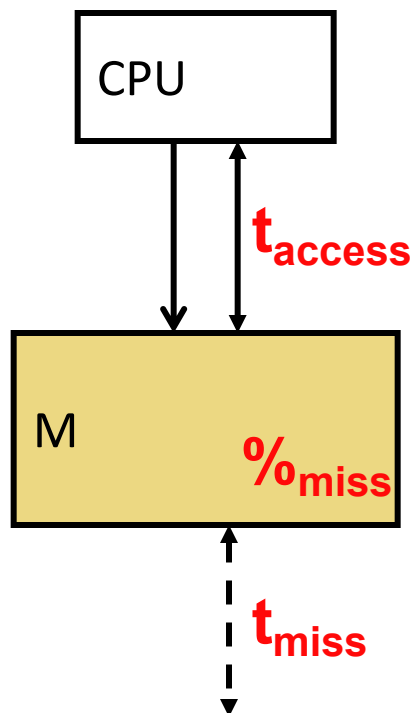
□ 上层Cache更看重 t_{access} (命中延迟)

- 访问非常频繁， t_{access} 更重要
- 未命中延迟(t_{miss})一般不会太长，所以未命中率 ($\%_{\text{miss}}$)不是特别重要
- 容量小、关联度低 (to reduce t_{access})
- block-size 偏小 (to reduce conflicts)

□ 下层Cache更看重 $\%_{\text{miss}}$ (未命中率)

- 访问不那么频繁， t_{access} 没那么重要
- 未命中延迟(t_{miss})很长，所以未命中率 ($\%_{\text{miss}}$)很重要
- 容量大，关联度高，block size偏大(to reduce $\%_{\text{miss}}$)

存储层次性能分析



□ 对于第M层cache

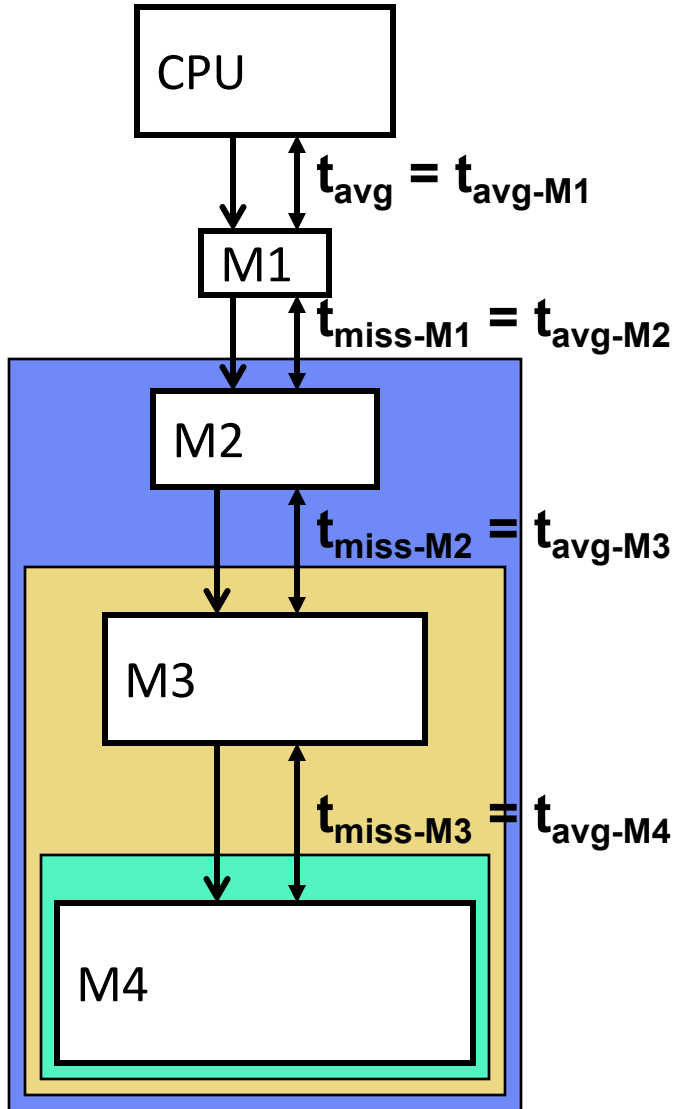
- **Access**: 读/写 M
- **Hit**: 数据命中
- **Miss**: 数据未命中
 - 必须从更低层cache读取
- **Fill**: 将data放入M
- $\%_{\text{miss}}$ (未命中率): $\# \text{misses} / \# \text{accesses}$
- t_{access} : 在cache中读取/写入数据的时间
- t_{miss} : 将数据读入M的时间(从下层获取)

□ 性能公式

- t_{avg} : 平均访问延迟

$$t_{\text{avg}} = t_{\text{access}} + (\%_{\text{miss}} * t_{\text{miss}})$$

存储层次性能分析



t_{avg}

t_{avg-M1}

$t_{acc-M1} + (\%_{miss-M1} * t_{miss-M1})$

$t_{acc-M1} + (\%_{miss-M1} * t_{avg-M2})$

$t_{acc-M1} + (\%_{miss-M1} * (t_{acc-M2} + (\%_{miss-M2} * t_{miss-M2})))$

$t_{acc-M1} + (\%_{miss-M1} * (t_{acc-M2} + (\%_{miss-M2} * t_{avg-M3})))$

...

分离Cache vs. 统一Cache

□ 分离 Cache: 指令和数据使用不同的cache (L1常用)

- 最小化结构冲突和命中延迟
- 统一式cache速度慢
- 可以分别对指令cache和数据cache进行优化
 - 指令cache不需要写

□ 统一 Cache: 指令和数据共用cache (L2, L3常用)

- 最小化为命中率
- + 减少capacity misses: 空间利用率更高
- 增加了conflict misses: 指令/数据冲突
- 指令/数据 结构冲突非常少: 同时发生指令/数据 miss时, 才会同时去读内存
- 进一步: 在多核处理器中多个核心的L2、L3 cache都是共用

Inclusion vs. Exclusion

□ Inclusion (包含)

- 将数据从内存先读入L2，再读入L1
 - 在L1中的block肯定在L2
- 如果L2中的block被替换, 必须同时从L1中剔除
 - 为什么? 在讲multicore的时候再深入讨论

□ Exclusion(排除)

- 将数据从内存读入L1而不是L2
 - L1中的block被替换的时候才会移到L2
 - Block在L1或者在L2 (不能同时存在)
- 适合L2相对于L1比较小的情况
 - Example: AMD's Duron 64KB L1s, 64KB L2

Cache 预取 (Prefetching)

预取 (Prefetching)

□ Idea: 提前取回程序需要的数据

□ Why?

- 访存时间长
 - 如果能够有效预取，可以大幅降低延迟
- 能够消除compulsory cache misses
- 能消除所有的cache misses么? Capacity, conflict?

□ 涉及到预测未来要访问哪些地址

- 适用于地址访问模式可预测的程序

预测和正确性

□ 错误的预取会影响程序正确性么？

- 不会

□ 不需要状态恢复

- 对比分支预测

预取的基础知识

□ 现代系统中, 预取通常以**cache line**为单位进行

□ 预取可以降低

- Miss rate
- Miss latency

□ 预取可以在不同层面完成

- Hardware
- Compiler
- Programmer
- System

预测的四个问题

□ What

- 预取哪些地址

□ When

- 何时发起预取动作 (early, late, on time)

□ Where

- 预取的数据放在哪里 (caches, separate buffer)
- 预取器本身放在哪里 (which level in memory hierarchy)

□ How

- 预取器如何操作，谁负责操作 (software, hardware, execution/thread-based, cooperative, hybrid)

预取的挑战：What

□ 预取哪些地址

- 预取没用的数据浪费资源
 - ✓ 内存带宽
 - ✓ Cache 或额外的缓存空间
 - ✓ 功耗
- 准确的预取非常重要

□ 如何知道预取哪些地址？

- 根据历史访问模式预测
- 利用编译器和程序员的信息

□ 预取哪些地址由预取算法决定

一些可预测的访问模式

□ Cache Block 地址

$A, A+1, A+2, A+3, A+4, \dots$

$B, B+42, B+84, B+126, \dots$

$C, C+2, C+5, C+9, C+11, C+14, C+18, C+20, C+23, C+27, \dots$

$X, Y, T, Q, R, S, X, Y, T, A, B, C, D, E, X, Y, T, F, G, H, X, Y, T, \dots$

$A+1, A+67, A+18, A+7, A+99, Z+1, Z+67, Z+18, Z+7, Z+99, P+1, P+67, P+18, P+7, P+99, \dots$

预取的挑战：When

□ 何时发起预取请求

- 预取的太早
 - ✓ 预取的数据可能会被换出
- 预取的太晚
 - ✓ 可能掩盖不住访存延迟

□ 数据什么时候预取影响预取器的实时性(timeliness)

□ 如何提高预取器的实时性

- **采用更积极的策略:** 比处理器访问数据的时间尽可能早(**硬件预取**)
- 预取指令放在访存指令更靠前的位置 (**软件预取**)

预取的挑战：Where (I)

□ 为哪一层cache预取？

- Memory to L4/L3/L2, memory to L1. 优点/缺点？
- L3 to L2? L2 to L1? (不同层之间独立的预取器)

□ 预取的数据放在哪里？

- 预取的数据和普通cache替换进来的数据一样对待？
- 预取的块并不知道是否能用得上

□ 替换的时候要向非预取的blocks倾斜么？

- 优先踢除预取的blocks

预取的挑战：Where (II)

□ 预取器放在哪里？

- 换句话说, 预取器可以获取哪些信息？
- L1 hits and misses
- L1 misses only
- L2 misses only

□ 获取更完整的访问模式:

- + 可以提高预取的accuracy和coverage
- 预取器需要观测更多的请求 (bandwidth intensive, more ports into the prefetcher?)

预取的挑战：How

□ 软件预取

- ISA 提供预取指令
- 程序员或编译器在代码中插入预取指令
- 只适用于比较规律的访问模式

□ 硬件预取

- 特殊硬件监测内存访问
- 记忆、查找、学习地址跨度/模式/关联
- 自动生成预取地址

预取的性能指标

- **Accuracy** (用到的预取数 / 总预取数)
- **Coverage** (预取的命中数 / 总内存访问数)
- **Timeliness** (及时的预取数 / 用到的预取数)

- **带宽需求**
 - 带有预取的带宽消耗 / 没有预取的带宽消耗

- **Cache 污染**
 - 因为预取导致的额外miss
 - 很难量化

软件预取 (I)

```
for (i=0; i<N; i++) {  
    __prefetch(a[i+8]);  
    __prefetch(b[i+8]);  
    sum += a[i]*b[i];  
}
```

```
while (p) {  
    __prefetch(p→next);  
    work(p→data);  
    p = p→next;  
}
```

```
while (p) {  
    __prefetch(p→next→next→next);  
    work(p→data);  
    p = p→next;  
}
```

Which one is better?

□访问模式有规律的情况非常高效. **问题:**

-- 预取指令占用带宽

- 提前多久预取? 很难决策

- 预取距离取决于硬件实现 (memory latency, cache size, time between loop iterations)

- 需要特殊的预取指令

-- 指针数据结构更加困难

软件预取 (II)

□ 编译器在哪里插入预取指令？

- 每个load指令都预取？
 - ✓ 带宽太高
- 分析代码决定哪些值得预取
 - ✓ 如果分析的代码不具有代表性怎么办？
- 提前多少条指令进行预取？
 - ✓ 对各种预取距离进行分析，并确定其使用概率
 - 通常需要提前很远地插入预取，以覆盖内存延迟

硬件预取

□ **Idea:** 专用硬件观察负载/存储访问模式，并根据过去的访问行为进行数据预取

□ **Tradeoffs:**

- + 可以根据系统实现进行调整
- + 不会浪费指令执行带宽
- 需要更多的硬件复杂性来检测模式
 - 在某些情况下，软件可能更高效

Next-Line 预取器

□ 最简单的硬件预取形式: 在每次内存访问后, 总是预取接下来的 N 个缓存行

□ Tradeoffs:

- + 实现简单
- + 顺序访问模式效果好 (instructions?)
- 特殊访问模式可能浪费带宽
- 即使是规律访问:
 - 如果 $\text{access stride} = 2$, 但是 $N = 1$, 准确率如何?
 - 如果访问是从高地址到低地址访问呢?

Stride 预取器

□ 考虑如下访问模式:

- ✓ $A, A+N, A+2N, A+3N, A+4N...$
- ✓ $\text{Stride} = N$

□ **Idea:** 记录连续内存访问之间的跨度；如果稳定，则用它来预测接下来的 M 次内存访问

更复杂的访问模式?

□ 简单访问模式

- Stride 预取器效果很好

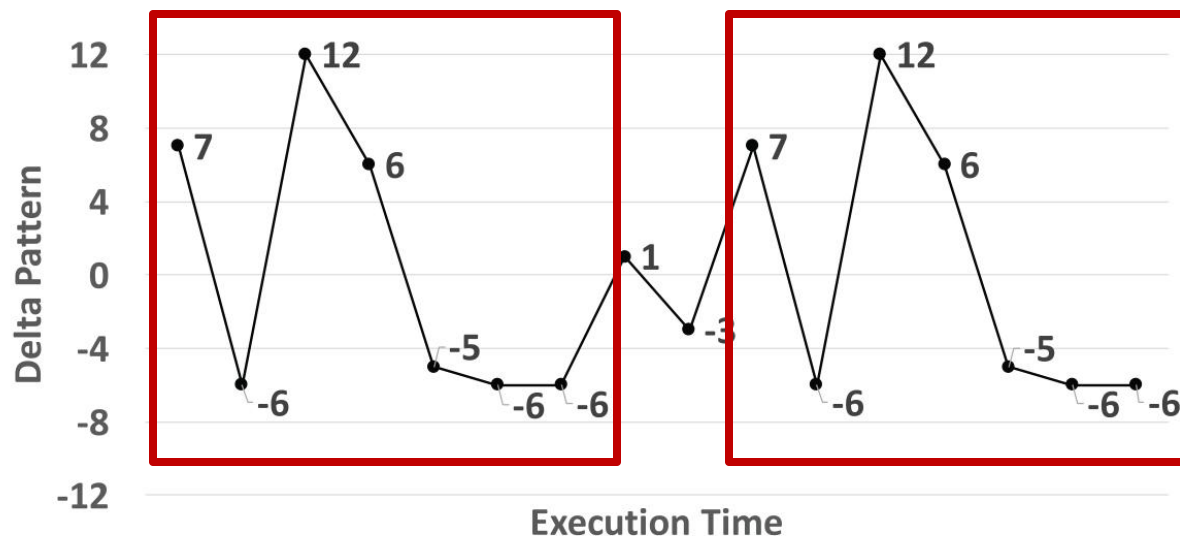
□ 复杂访问模式

- ✓ E.g., multiple regular strides
- ✓ +1, +2, +3, +1, +2, +3, +1, +2, +3, ...

□ 无规律访问模式

- 链表遍历
- 间接数组访问
- 随机访问
- 多个数据结构并发访问
- ...

复杂访问模式检测



复杂但是可预测的访问模式

Path Confidence based Lookahead Prefetching

Jinchun Kim*, Seth H. Pugsley[†], Paul V. Gratz*, A. L. Narasimha Reddy*, Chris Wilkerson[†] and Zeshan Chishti[†]

*Texas A& M University

cienlux@tamu.edu, pgratz@gratz1.com, reddy@tamu.edu

[†]Intel Labs

{seth.h.pugsley, chris.wilkerson, zeshan.a.chishti}@intel.com

空间内存流方法 (SMS)

□ 如果检测不到规律的stride呢？

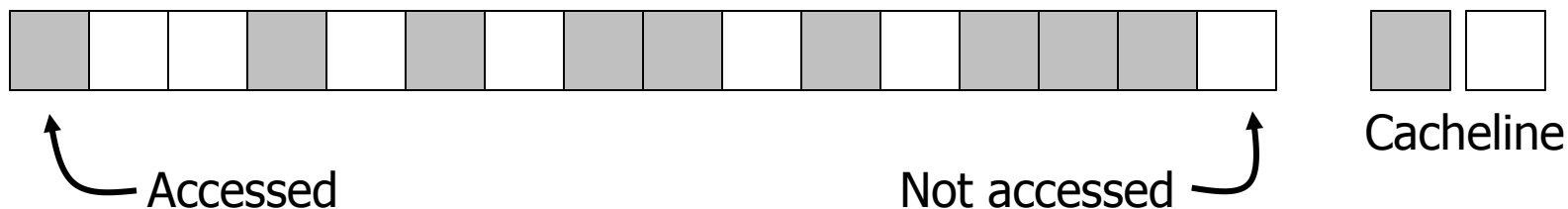
- ✓ 乱序执行的内存访问
- ✓ 链表遍历
- ✓ ...

□ 空间内存流方法 (Spatial Memory Streaming) :

- ✓ 将访问模式描述为一个大的地址空间（例如物理页面）上的位图模式，而不是连续stride的序列
- ✓ 记录并学习模式重复规律

空间内存流方法 (SMS)

PC_x 按照如下bit模式访问page P₁



PC_x 按照如下bit模式访问page P₂



Pattern learning: PC_x →



虚拟内存

(Virtual Memory)

内存虚拟化

□ 多个进程如何共享内存?

- **目标:** 每个进程以为自己拥有全部内存

□ 一个进程可能需要比实际物理内存更多的内存空间

- 需要加载的指令/数据比物理内存更大
- 需要内存表现的像一个cache
 - 磁盘作为下一级存储 (slow)
 - Write-back, write-allocate, large blocks or “pages”

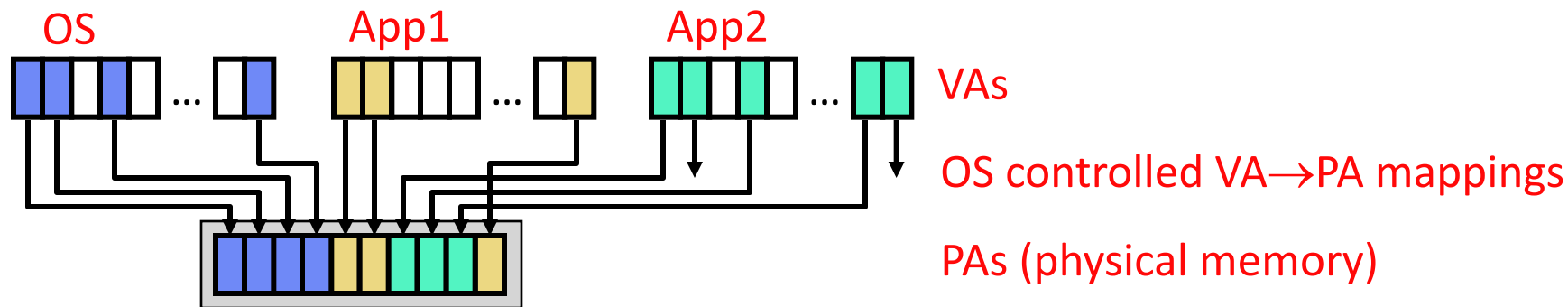
□ 解决方案:

- Part #1: 将内存看作一个 “cache”
 - 将溢出的数据块存在磁盘上的 “交换” 空间
- Part #2: 增加一级映射 (地址转换)

虚拟内存 (Virtual Memory)

□ 虚拟内存 (VM):

- Level of indirection
- 应用产生的地址为虚拟地址 (VAs)
 - 每个进程以为自己拥有 2^N 个字节的地址空间
- 内存访问使用物理地址 (PAs)
- 虚拟地址到物理地址的转换以page为单位(粗粒度)
- 操作系统负责虚拟地址到物理地址的映射(页分配)
- 逻辑上: 地址转换需要在每次取指令和访问内存数据之前进行
- 实际上: 有很多硬件加速措施提高转换速度



虚拟内存 (Virtual Memory)

□ 程序使用**虚拟地址 (VA)**

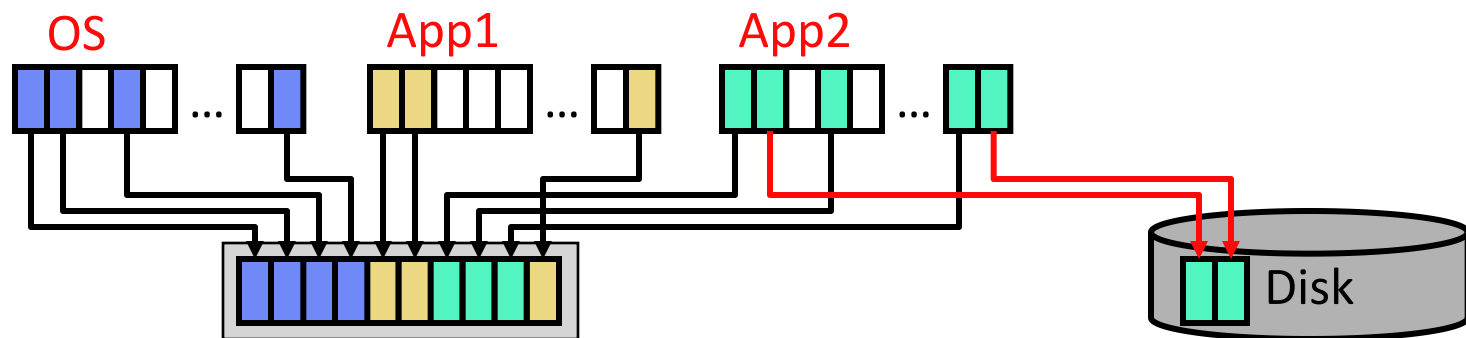
- 虚拟地址位数为N bits (指针大小, 现在通常为64 bits)

□ 内存使用**物理地址 (PA)**

- 物理地址位数为M bits, M通常小于N
- 2^M 表示能支持最大的物理内存
- 现代计算机一般采用48 bits, Intel/AMD正在尝试56 bits

□ 虚拟地址→物理地址转换以**page**为粒度

- 映射不需要保持物理页连续
- 虚拟页可能没有映射到任何物理页
- 没有映射的虚拟页可能在磁盘上(换出去了)或者还未被使用过



可看作是全相联 + 软件替换的cache

- 内存的 t_{miss} (未命中延迟)很高: 降低未命中率至关重要

Parameter	I\$/D\$	L2	Main Memory
t_{hit}	2ns	10ns	30ns
t_{miss}	10ns	30ns	10ms (10M ns)
Capacity	8–64KB	128KB–2MB	64MB–64GB
Block size	16–32B	32–256B	4+KB
Assoc./Repl.	1–4, LRU	4–16, LRU	Full, “working set”

虚拟内存的好处

❑ 多程序之间隔离

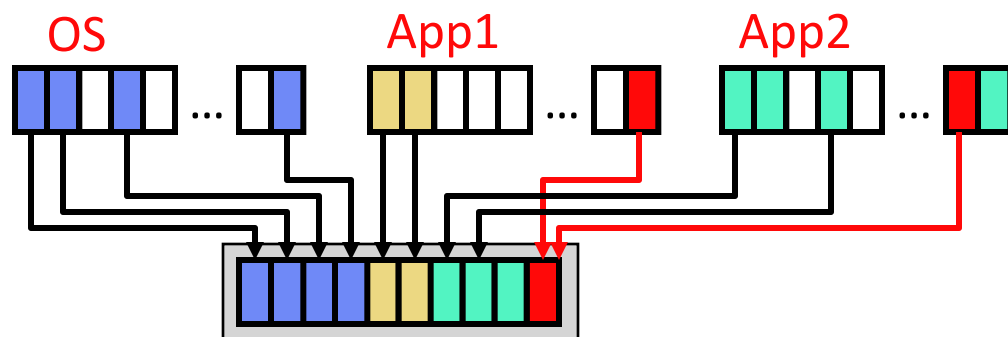
- 每个程序以为自己有 2^N 大小的内存空间
- 防止程序互相访问彼此的内存空间
 - 无法知道其他程序的物理地址!

❑ 安全保护

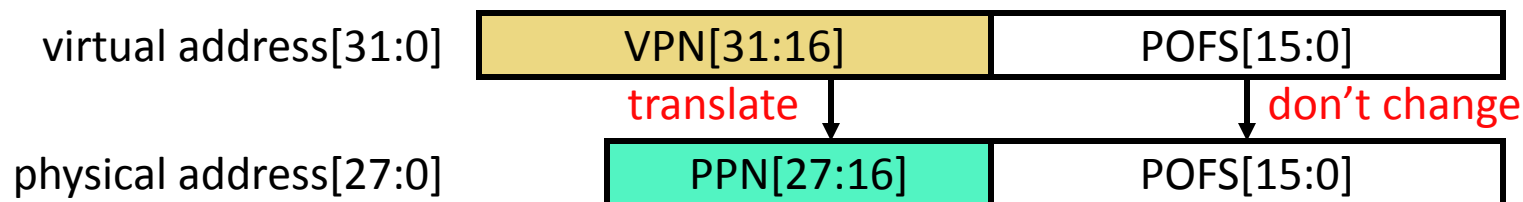
- 每个页有读/写/执行的权限 (OS设置)
- 由硬件保证

❑ 进程之间通信

- 将同一个物理页映射到多个虚拟地址空间
- 或者通过UNIX **mmap()**共享文件



地址转换



□ 虚地址到物理地址的映射称为地址转换

- 将虚地址分成**虚拟页号 (VPN)** & **页内偏移 (page offset)**
- 将虚拟页号转换为**物理页号 (PPN)**
- 页内偏移不需要转换

□ 例如

- 页大小为64KB → 16-bit 页内偏移
- 32-bit计算机 → 32-bit虚拟地址 → 16-bit 虚拟页号
- 最大256MB物理内存 → 28-bit 物理地址 → 12-bit 物理页号

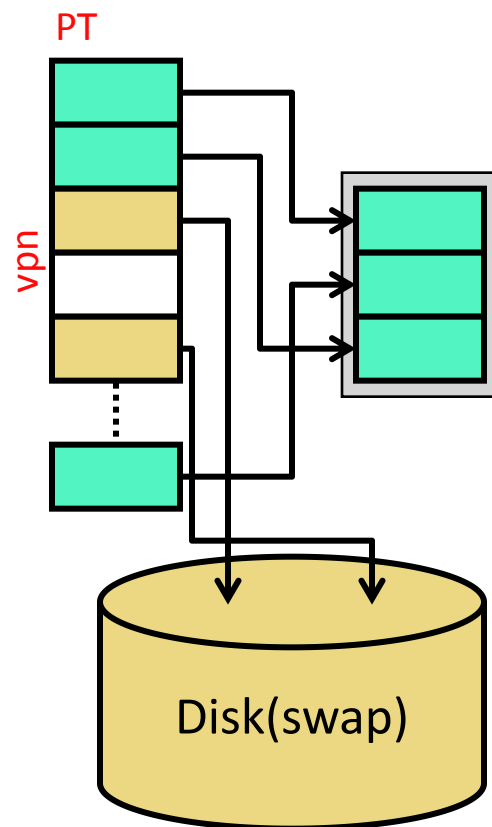
地址转换机制 I

□ 地址如何转换?

- 软硬件协同完成

□ 每个进程有一个页表(page table)

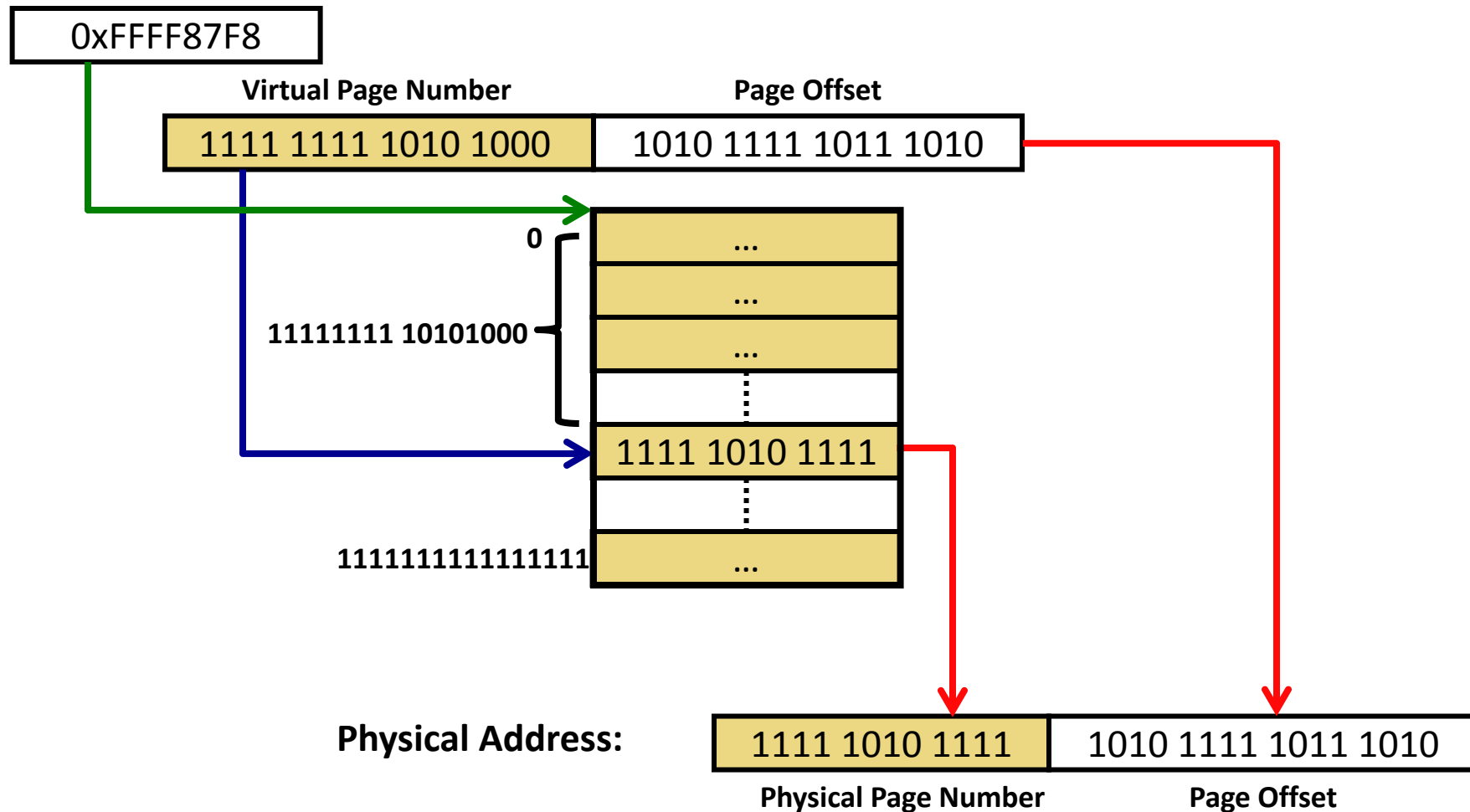
- 由操作系统维护的数据结构
- 将虚拟页映射到物理页或磁盘地址
 - 如果页从未被访问过, 虚拟页对应的项为空
- 地址翻译就是查页表



页表示例

Example: 内存访问地址为 0xFFA8AFBA

页表起始地址



页表大小

□ 以下计算机的页表大小如何计算？

- 32-bit 计算机
- 4B 页表项大小 (PTEs)
- 4KB 页大小



- 32-bit 计算机 → 32-bit 虚拟地址 → $2^{32} = 4\text{GB}$ 虚拟内存
- 4GB 虚拟内存 / 4KB 页大小 → 1M 虚拟页
- 1M 虚拟页 * 4 Bytes/每页表项 → 4MB

□ 如果页大小为 64KB 呢？

□ 如果是 64-bit 计算机呢？

页表可能很大

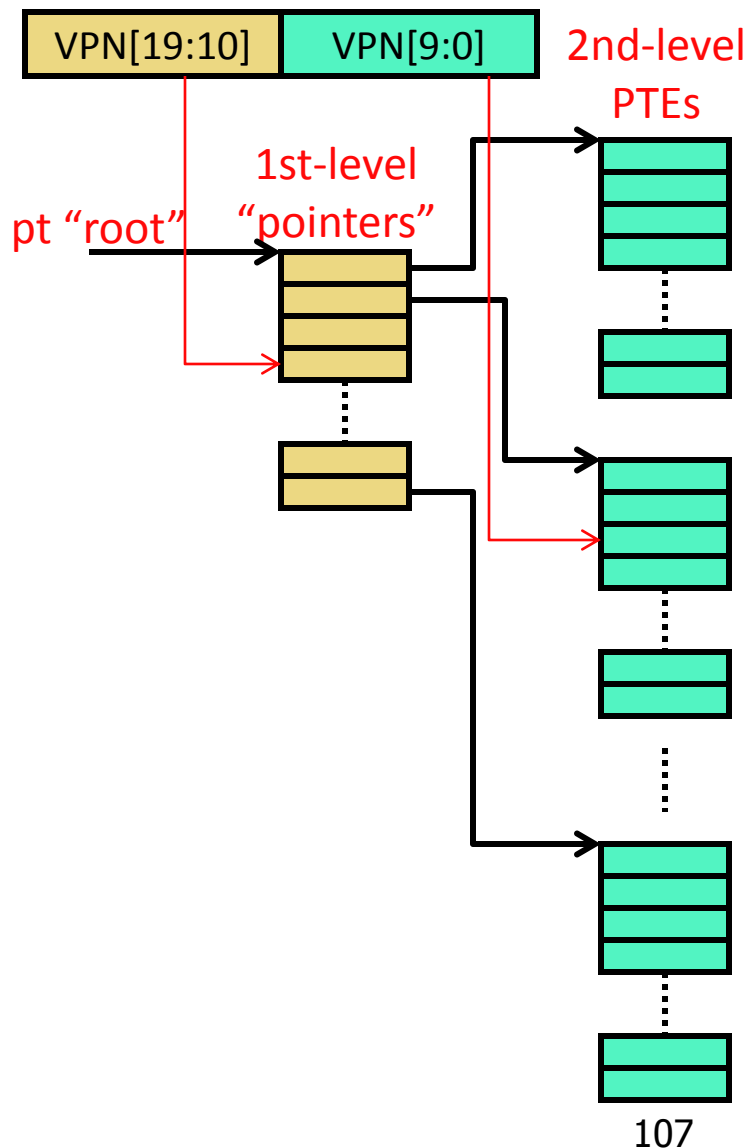
多级页表

□ 多级页表

- 将页表变为树形结构
- 最下层保存页表项(PTEs)
- 上层页表保存到下层的指针
- 虚拟页号的不同部分用于索引不同的层

□ 20-bit 虚拟页号

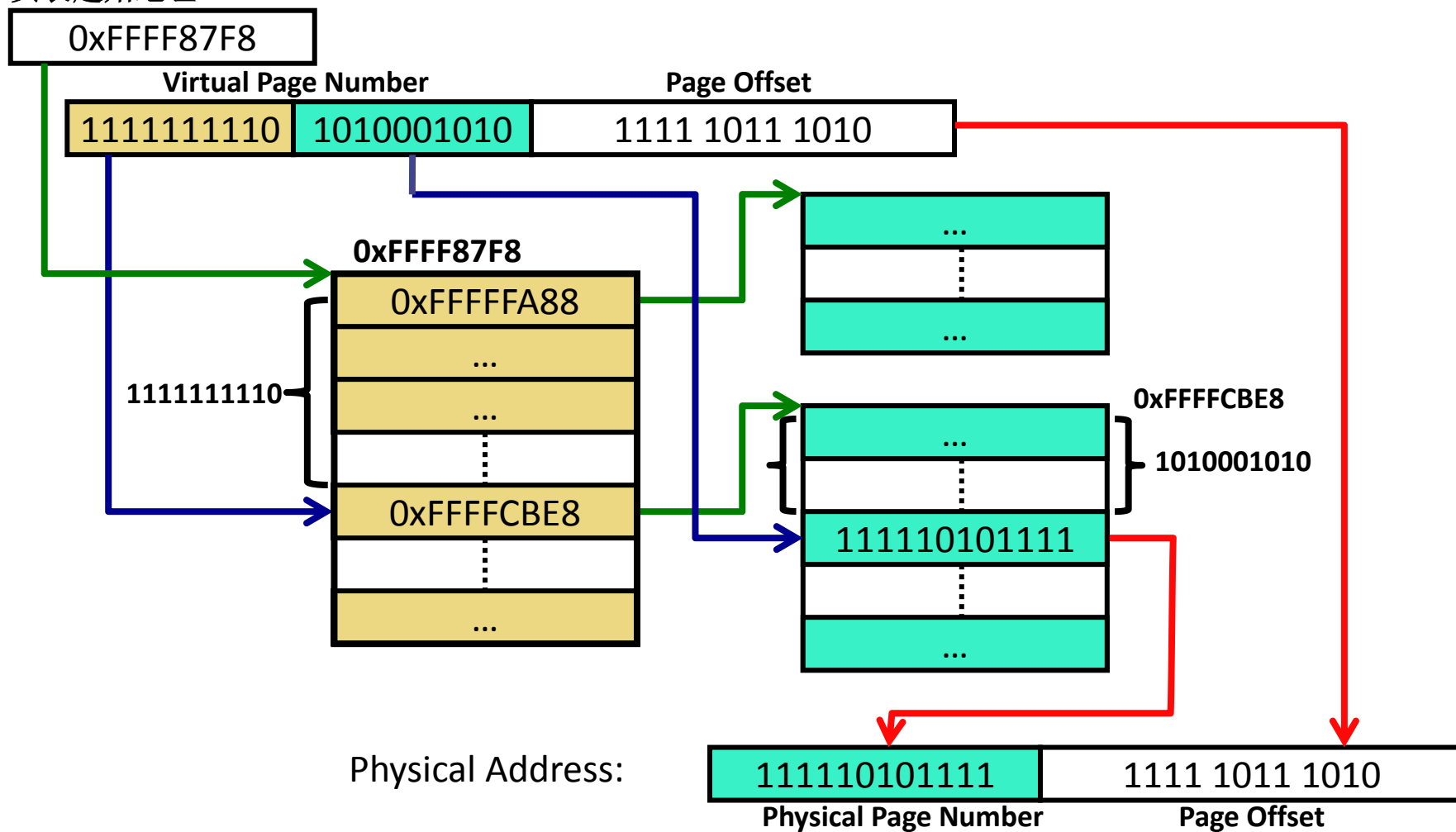
- 高位10 bits索引第1层
- 低位10 bits索引第2层
- 实际中, 通常大于2层



多级地址转换

Example: 内存访问地址 0xFFA8AFBA

页表起始地址



多级页表

□ 多级结构节省空间了么?

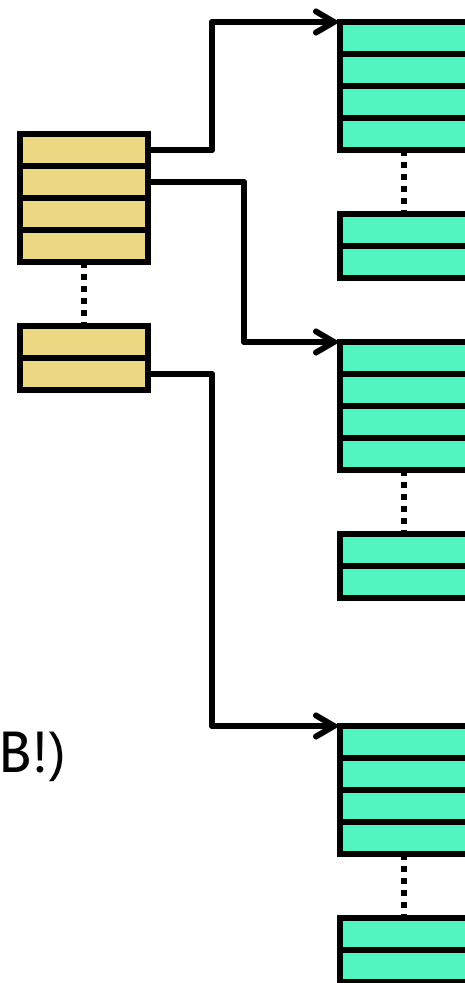
- 第2级表的所有页表项加起来并没有减少 (i.e., 4MB)?
- 是, 但是...

□ 大量虚拟地址空间不会被访问

- 相应的第2级表不会被分配
- 第1级表相应的表项为空

□ 如果使用了连续的256MB空间,页表多大?

- 每个第2级表覆盖4MB的虚拟地址空间
- 使用1个第1级表 + 64个第2级表
- 一共使用65个表*4KB/表 = 260KB (远小于4MB!)



□ 很多ISAs支持多种页大小

- x86: 4KB, 2MB, 1GB

□ 大页的优缺点

- + 减少页表空间
 - 表项变少了
- + 页表层数更少
 - 查找速度快
- 页内空间浪费更严重
 - 分配 2MB 大小的页但只用了 5KB
- 实现更复杂
 - OS 会寻求其他措施提高page利用率

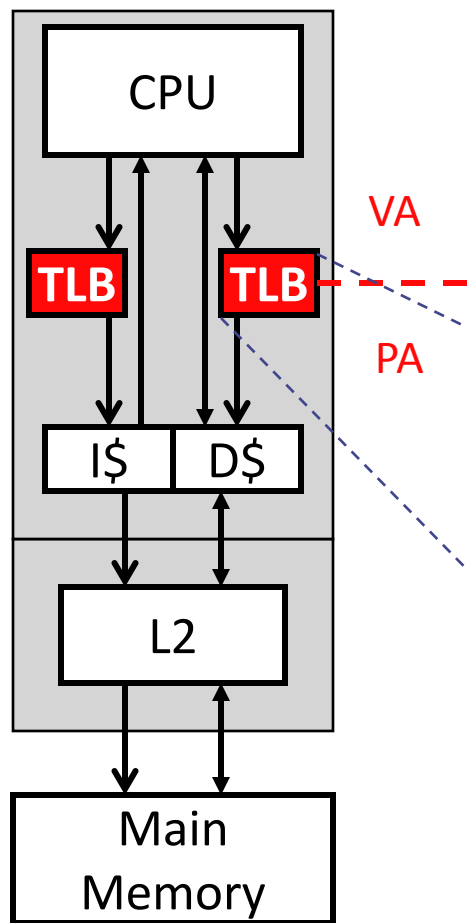
地址转换

□ 概念上

- 地址转换需要在每一次内存访问(cache访问)之前进行
- 每次转换都需要访问页表
 - 效率非常低

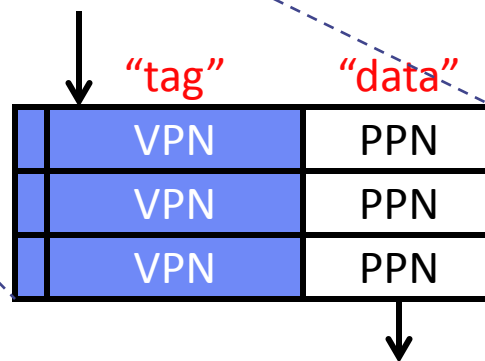
□ 现实中

- **Translation Lookaside Buffer (TLB)**: 缓存转换结果
- 只有在TLB未命中的时候才去访问页表

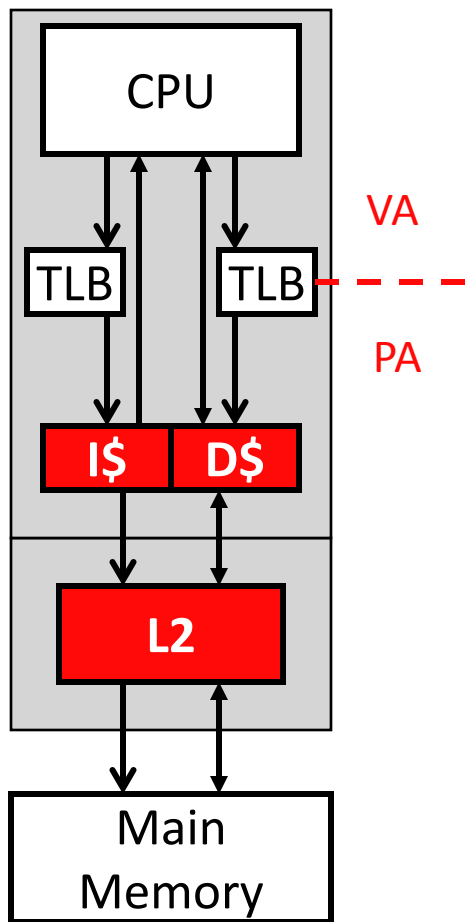


- **Translation lookaside buffer (TLB)**

- 一个小cache: 16–64表项
- 关联度 (高于4, 或者全相联)
- + 利用页表访问的时间局部性
- 如果TLB未命中?
 - 调用未命中处理程序, 遍历页表



TLB & Cache



❑ Caches

- 物理地址作为索引和tag
- + 好处
 - 自然实现隔离(不同任务的物理地址不同)
 - 上下文切换(context switches)无需flush
- + 可以实现基于cache的进程间通信
 - 一个cache block可由多个进程共享
- 慢: 即使TLB命中仍需要增加一个周期

❑ TLBs

- 虚拟地址作为index和tag
- 上下文切换(context switches)需要flush

TLB Miss

□ **TLB miss:** 地址转换结果不在TLB, 在页表

- 两种处理方式，都相对比较快

□ **基于硬件的方式:** e.g., x86, recent SPARC, ARM

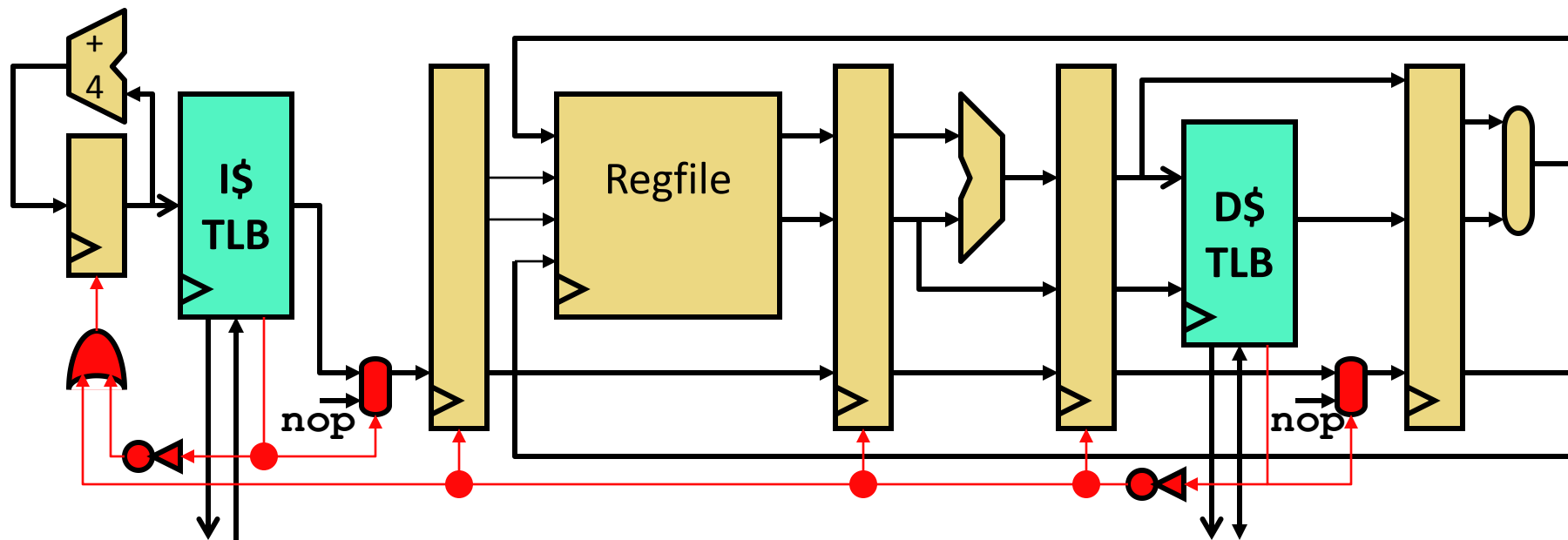
- 页表的基地址由寄存器保存，硬件负责遍历页表取回数据
- + 延迟: 避免了操作系统程序 (avoids pipeline flush)
- 页表格式必须是硬编码的

□ **基于软件的方式:** e.g., Alpha, MIPS

- + 采用简短的（大约10条指令）操作系统程序遍历页表并更新TLB
- + 页表格式可以灵活定义
- 延迟: 读了1-2次内存访问 + OS call (pipeline flush)

□ **当前基于硬件的方式更受欢迎**

TLB Miss 和流水线停顿



□ TLB 未命中会导致流水线停顿(像数据危害一样)

- 如果TLB是基于硬件管理的

□ 如果TLB是软件管理的

- 需要产生中断
- 硬件不负责处理TLB 未命中

缺页异常(Page Faults)

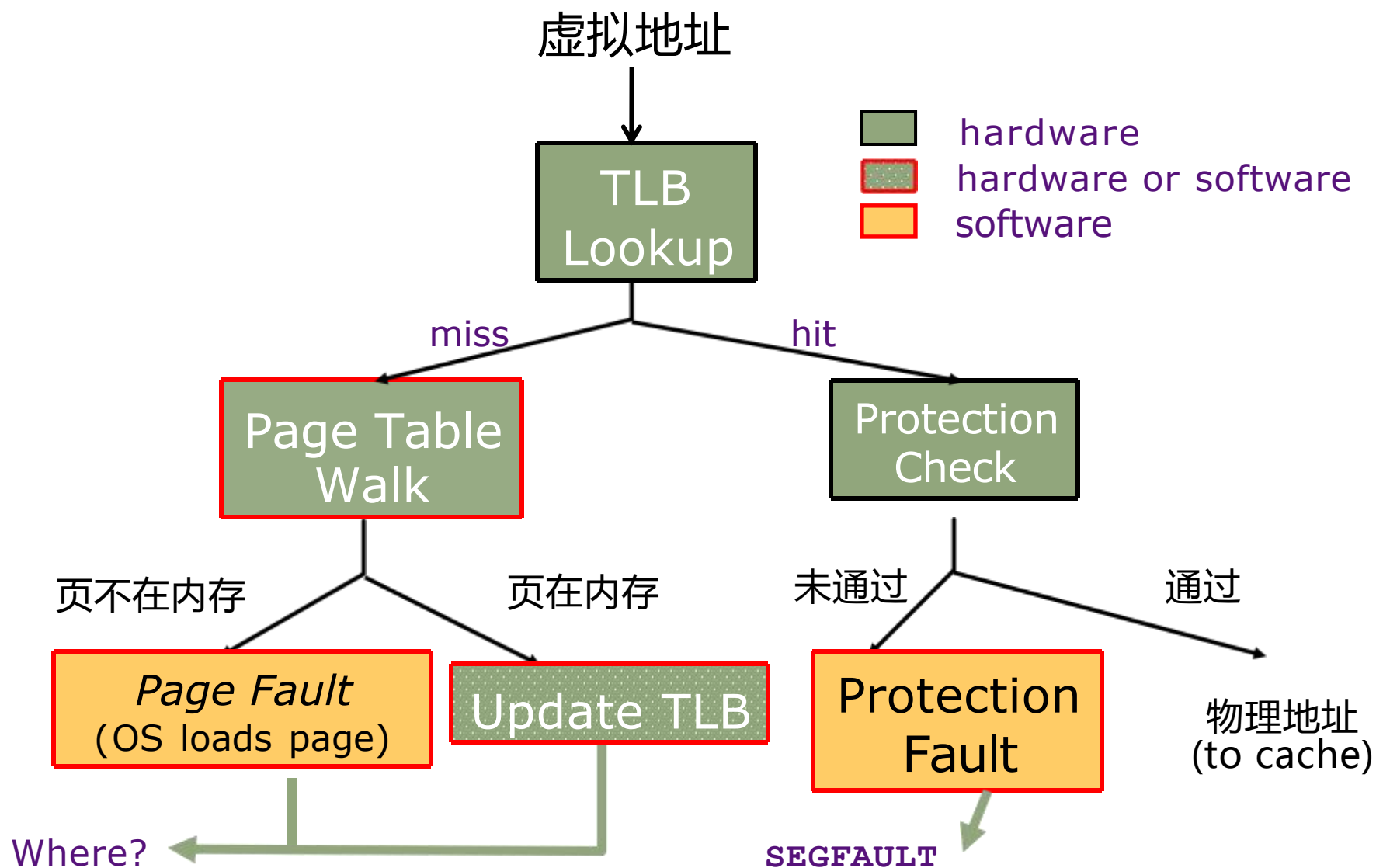
□ Page fault: PTE访问页不在TLB和页表中

- 页不在主存中(可能在磁盘, 或还没有分配)
- 如果页还没有分配 → segmentation fault

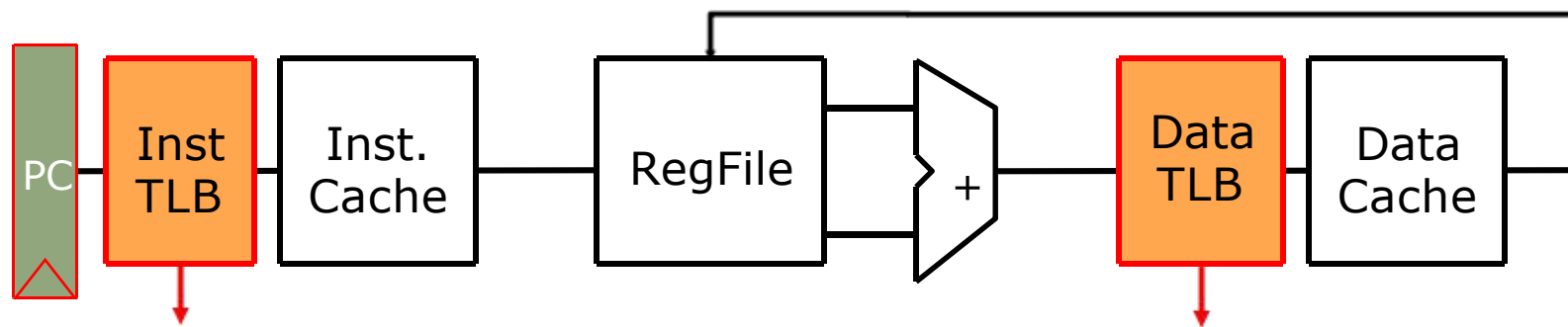
□ OS 要做的事:

- 选择一个要替换掉的物理页
- 如果要替换的页是“脏”页, 需要先写回磁盘
- 从磁盘读入缺失的页
 - ✓ 延迟很长 (~10ms), OS 调度其他任务
- 更新页表, 刷新TLBs, 重试内存访问操作

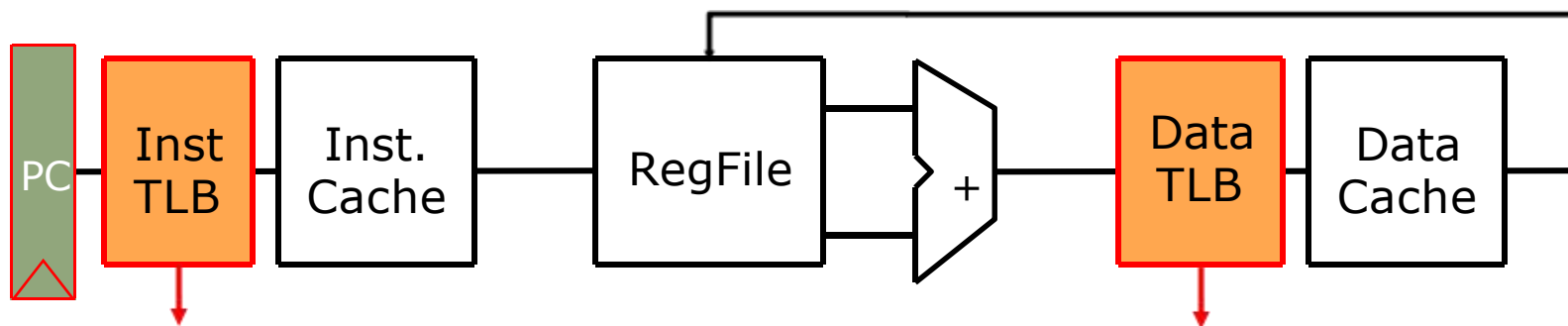
地址转换：整体流程



地址转换过程优化

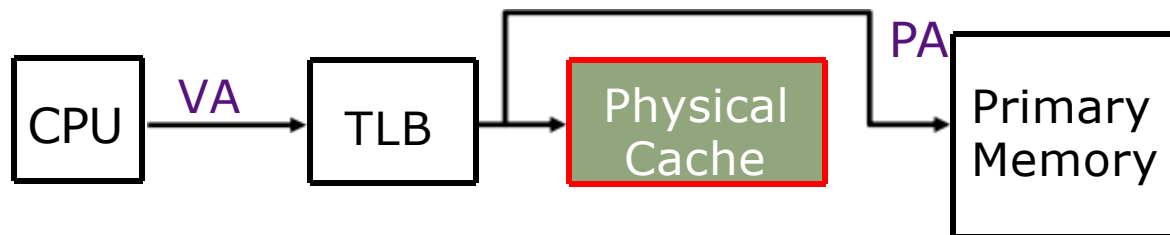


地址转换过程优化

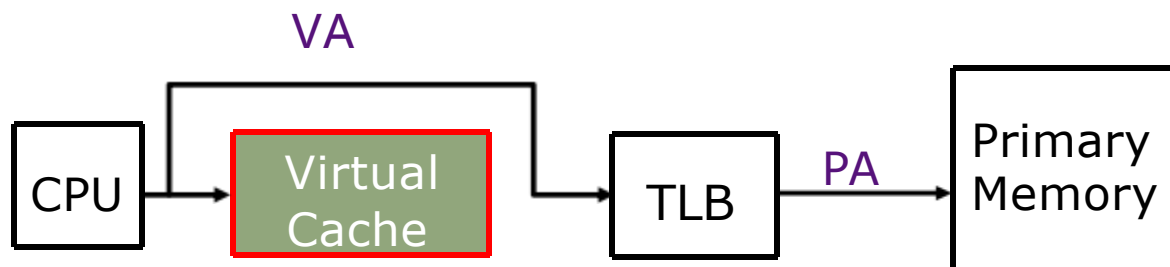


- ❑ 目前地址转换为串行，先访问TLB，再访问Cache
- ❑ 如何优化地址转换过程，降低延迟？
 - ✓ 虚拟地址cache
 - ✓ TLB/Cache并行访问

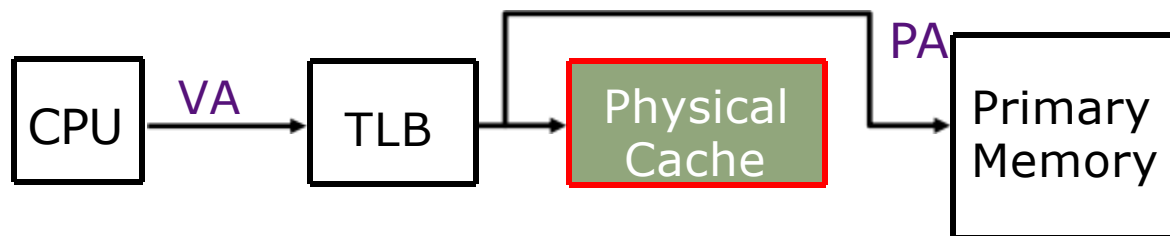
虚拟地址Cache



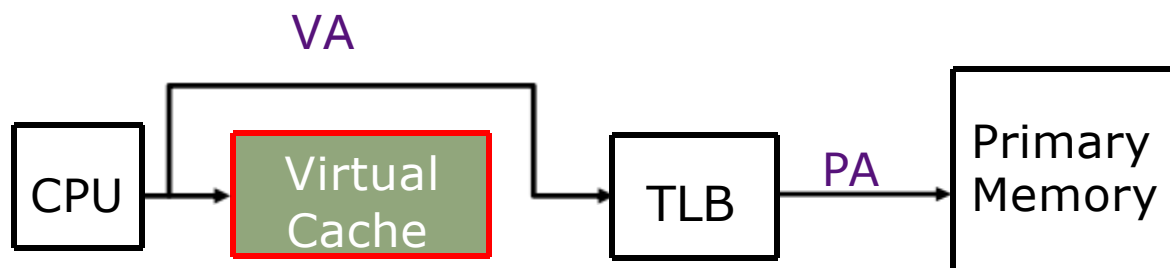
替代方案: 将cache放在TLB之前



虚拟地址Cache

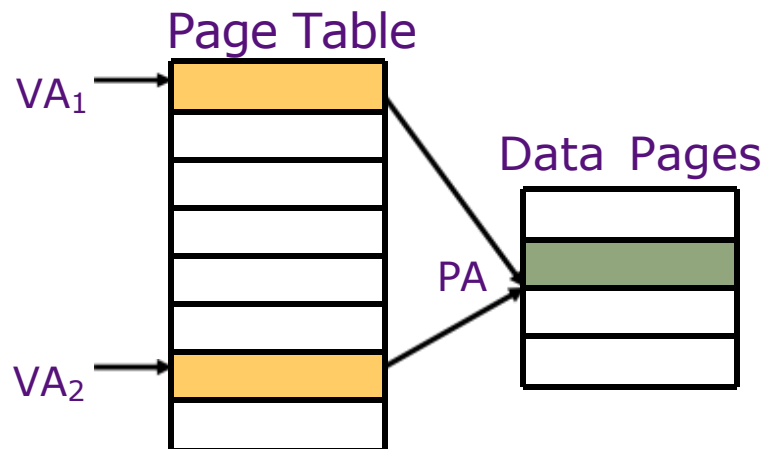


替代方案: 将cache放在TLB之前



- 如果命中内存访问一步就完成 (+)
- 发生上下文切换需要清空cache，除非增加进程标识 (-)
- 共享页会导致*aliasing*问题 (-)

虚地址cache引起的Aliasing



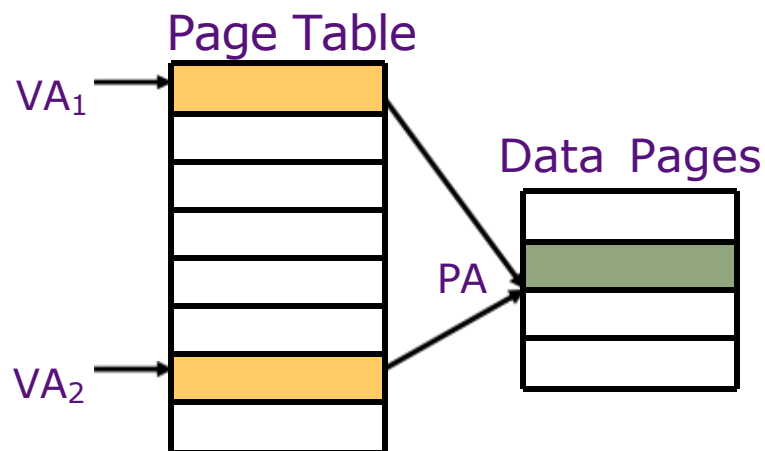
两个虚拟地址共享
同一个物理页

Tag	Data
VA ₁	1st Copy of Data at PA
VA ₂	2nd Copy of Data at PA

- 导致同一块数据在cache中有两个副本
- 写其中一个副本另外一个副本不知道，产生不一致!

因为cache按照虚地址访问，访问VA1时会把数据块放入cache的某个位置，访问VA2时又会将同一个数据块放入cache的另外一个位置，所以同一个数据块有两个copy

虚地址cache引起的Aliasing



Tag	Data
VA ₁	1st Copy of Data at PA
VA ₂	2nd Copy of Data at PA

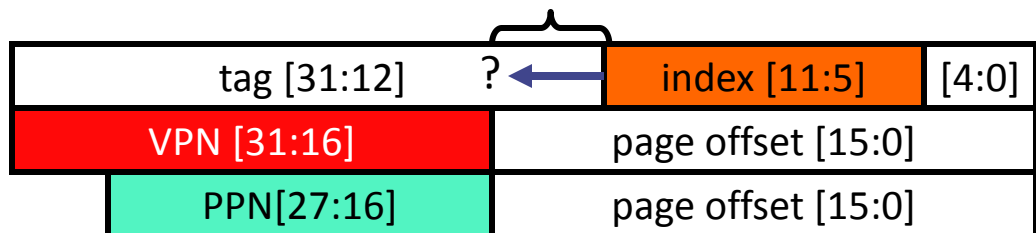
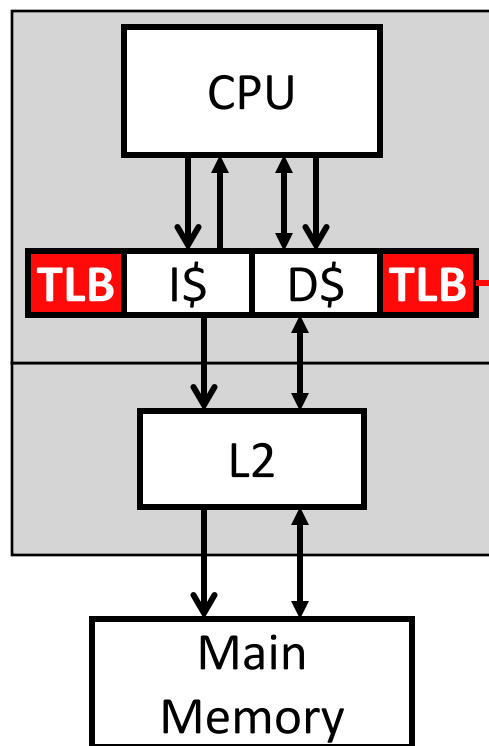
- 导致同一块数据在cache中有两个副本
- 写其中一个副本另外一个副本不知道，产生不一致!

常用解决方案：不允许多个副本同时存在

对于采用直接映射的cache

- 要求共享物理页的虚拟地址中的index bits必须相同，这样就会映射到同一个cache block，不会产生多个副本 (early SPARCs)

TLB & Cache 并行访问



□ 并行访问的条件

- **(cache size) / (associativity) ≤ page size**
- Index bits在物理地址和虚拟地址中相同!

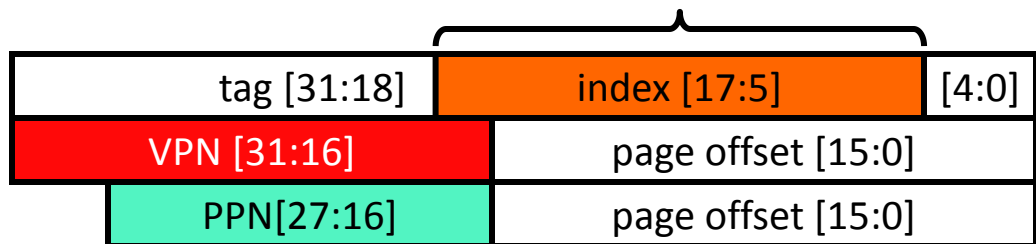
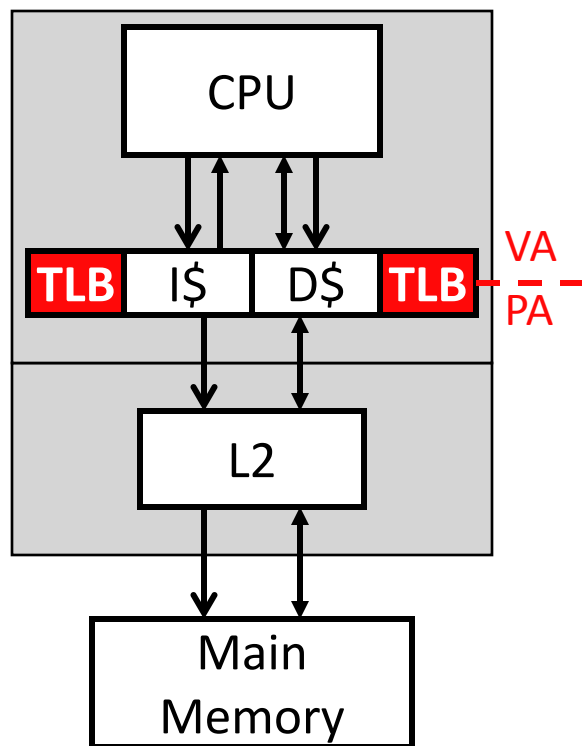
□ 并行访问TLB和Cache

- Cache 访问在后期才需要比较tag
- + 快: 访问TLB时可以同时访问Cache(完成set定位)
- + 无上下文切换问题
- 当今主流做法

□ Index bits 限制了cache容量

- 如果index太长, 会和VPN重叠
- 可以通过增加关联度缓解

TLB & Cache 并行访问



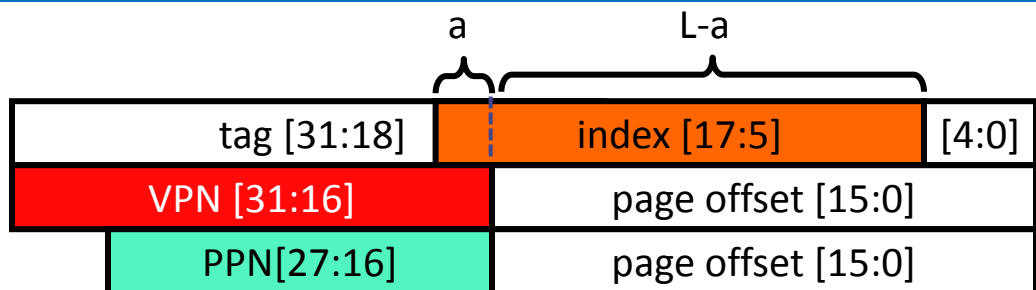
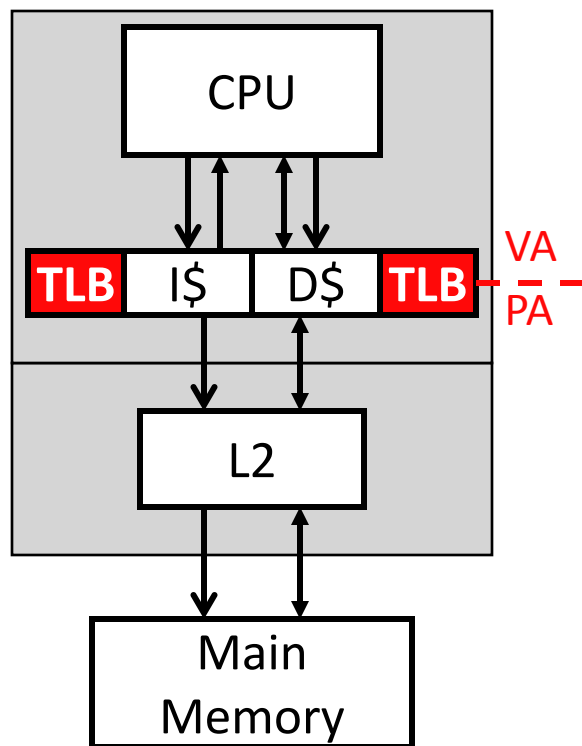
□ 如果index太长？

- **(cache size) / (associativity) > page size**
- Index bits在物理地址和虚拟地址中可能不同!

□ 导致的问题 -> Aliasing

- 假设两个不同的虚拟地址V1, V2共享一个物理页(转换后的物理地址相同)
- V1和V2的index bits可能不同, 导致数据映射到cache不同的block(同一物理地址的数据在cache中有两个不同的副本)

TLB & Cache 并行访问



□ 如果index太长？

- **(cache size) / (associativity) > page size**
- Index bits在物理地址和虚拟地址中可能不同!

□ 解决方案:virtual-index physical-tag cache

- cache同时查找 2^a 个block，index由两部分构成：低位有 $(L-a)$ bits，直接来自虚拟地址；高位有 a bits，分别为 $2^0, 2^1, \dots, 2^{a-1}$
- TLB和cache并行访问
- 转换成物理地址后，将物理地址tag和 2^a 个block中的tag都进行对比

虚拟内存现状

□ 个人电脑/服务器/手机处理器采用完全按需分页的虚拟内存

- 不同内存大小的计算机之间可移植
- 多用户和多任务之间的安全保护
- 多任务共享物理内存

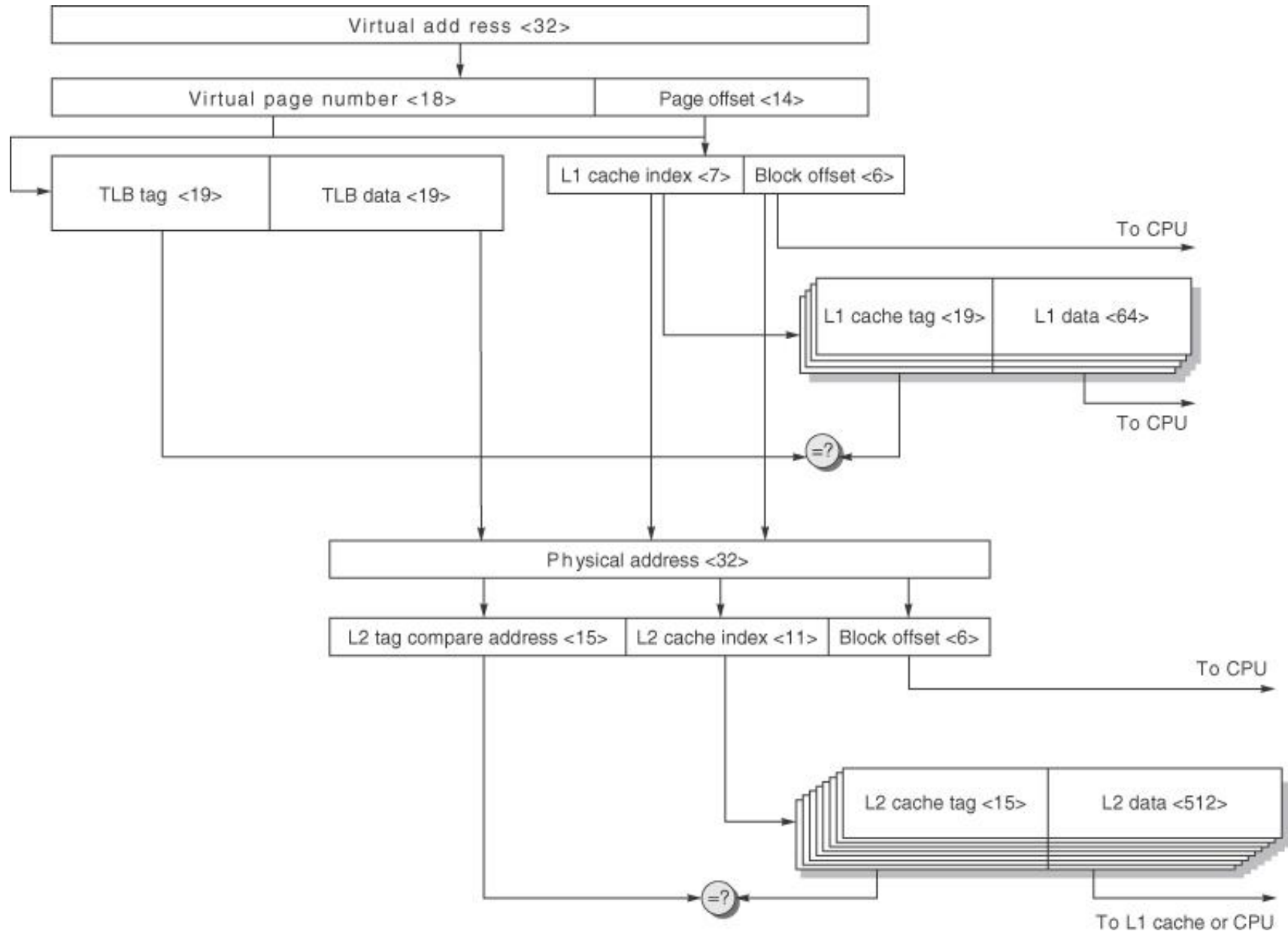
□ 超算和GPU有地址转换和保护，但不是按虚分页

- 让任务数据全部放在内存，节省换入换出时间
- 大部分任务是批处理模式（一批任务正好能放在内存）

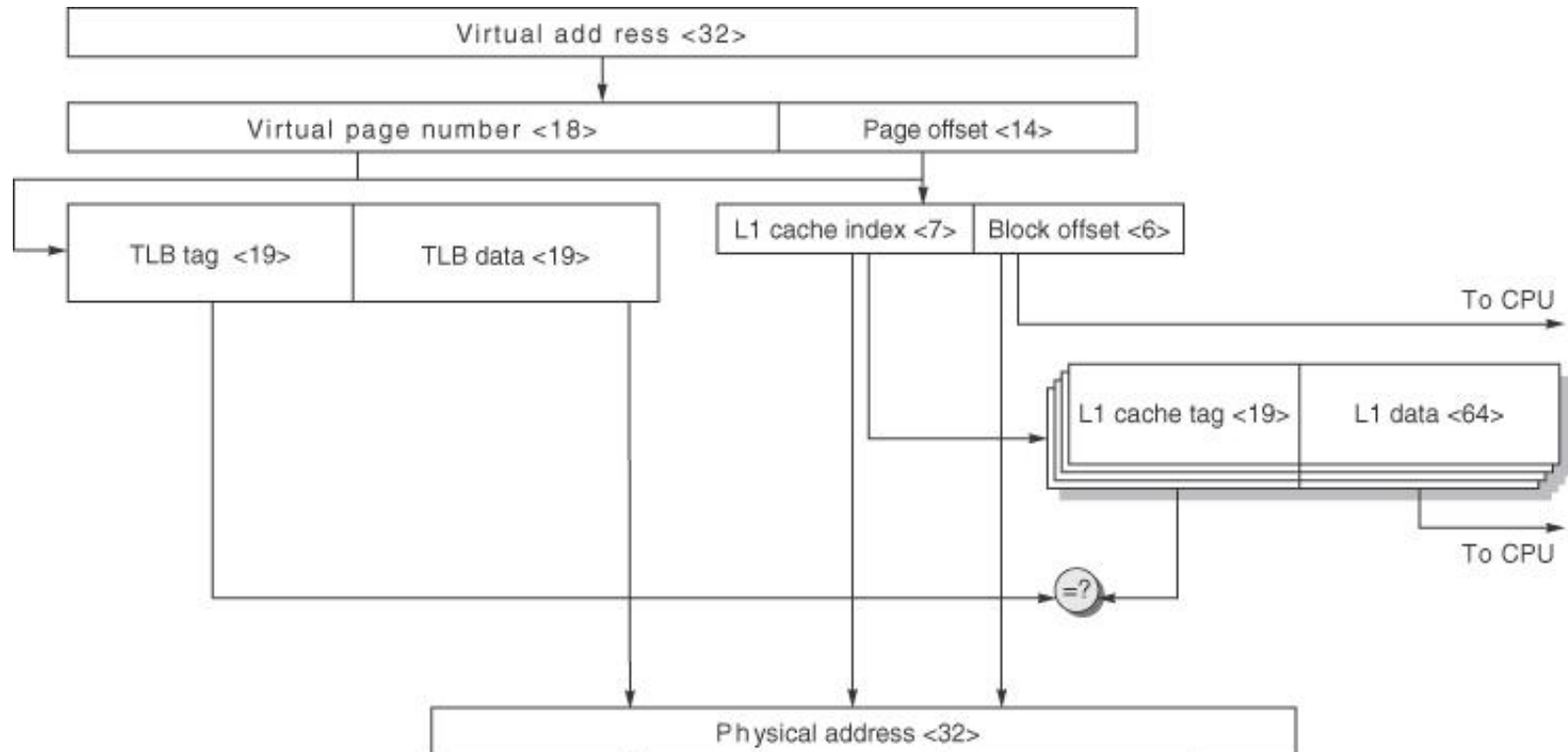
□ 部分嵌入式设备和DSPs仅支持物理寻址

- 虚拟内存带来的硬件开销太大
- 通常没有二级存储用来放换出的数据！
- 程序都是根据产品特定的内存配置定制化编写

示例：AMR Cortex-A8 data caches/TLP



示例：AMR Cortex-A8 data caches/TLP



TLB 采用全相联，有32个blocks. L1 cache采用4路组相联，block大小为64-byte，容量为32 KB. L2 cache采用8路组相联，block大小为64-byte，空间为1 MB.

